



S.E.P. TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO de Tuxtepec

ACTIVIDAD:

PROYECTO JUEGO

PRESENTAN:

LURIA SANTIAGO DANIEL 22350722

DOCENTE:

JULIO AGUILAR CARMONA

CARRERA:

INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA:

CIENCIAS DE DATOS

05/JUNIO/2026



Contenido

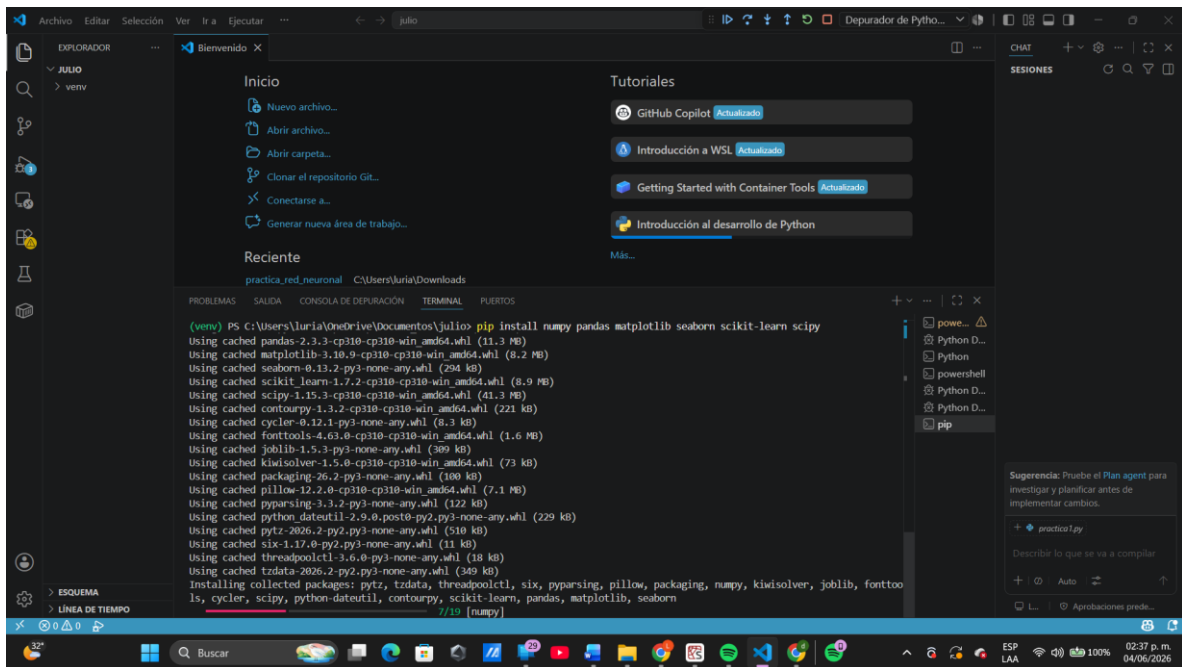
Preparación del entorno	5
Instalación de librerías.....	5
Práctica 2: Análisis de Calidad Industrial (Industria 4.0)	6
1. Contexto Profesional.....	6
2. Objetivo	6
3. Requerimientos.....	6
4. Tareas a realizar.....	6
Fase A: Análisis Exploratorio (EDA)	6
Fase B: Agrupación y Segmentación	6
Fase C: Visualización e Interpretación	6
5. Entregable	7
Práctica 3: Análisis de Eficiencia en Logística Global	14
1. Contexto Profesional.....	14
2. Objetivo	14
3. Requerimientos.....	14
4. Tareas a realizar por el alumno	14
Fase 1: Calidad de Datos	14
Fase 2: Análisis de Relaciones	14
Fase 3: Comparativa de Modalidades.....	14
Fase 4: Visualización	15
5. Entregable	15
Práctica 4: Reducción de Dimensionalidad en Monitoreo Industrial	23
1. Contexto Profesional.....	23
2. Objetivo	23
3. Requerimientos.....	23
4. Instrucciones para el Estudiante.....	23
Fase 1: Preparación	23
Fase 2: Ejecución de PCA	23

Fase 3: Selección de Componentes.....	23
Fase 4: Visualización	23
Práctica 5: Reducción de Dimensiones en Tráfico de Red (Cyber-Systems)	40
Unidad 2: Tratamiento de Datos – TecNM	40
Contexto Profesional	40
Paso 1: Generación y Exploración de Datos.....	40
Paso 2: Estandarización (Scaling)	40
Paso 3: Análisis de Varianza (Varianza Explicada)	41
Paso 4: Interpretación de Cargas (Loadings)	41
Paso 5: Visualización (Biplot)	41
title: "Práctica 6: Optimización de Sensores en Smart Cities" subtitle: "Unidad 2: Tratamiento de Datos - TecNM" author: "Guía Paso a Paso con Interpretación" output: html_document: toc: true theme: united	50
Contexto Profesional	50
Paso 1: Generación de Datos Urbanos	50
Paso 2: Estandarización de las Métricas	51
Paso 3: Análisis de Varianza Acumulada	51
Paso 4: Interpretación de los Componentes (Loadings)	51
Paso 5: El Biplot de Diagnóstico Urbano	52
Práctica 7: Implementación de Modelos de Machine Learning.....	60
Unidad 3: Modelado - Ciencia de Datos (TecNM)	60
Contexto Profesional	60
Paso 1: Preparación del Dataset.....	60
Paso 2: Regresión Lineal (Predecir Salario)	61
Paso 3: Clasificación KNN (Predecir Retención).....	61
Paso 4: Clustering No Supervisado (K-Means)	61
Práctica 8: Clasificación de Intrusiones con KNN y Logística	70
Unidad 3: Modelado - TecNM.....	70
Contexto Profesional	70
Paso 1: Simulación de Tráfico de Red	70

Paso 2: Modelo de Regresión Logística	70
Paso 3: Clasificación con KNN	70
Paso 4: Análisis de Resultados (Clustering K-Means)	71

Preparación del entorno

Instalación de librerías



The screenshot shows the Visual Studio Code interface with a terminal window open. The terminal is running a command to install various Python libraries using pip. The output shows the installation progress for each library, including the version and the file being downloaded or cached. The libraries being installed are: numpy, pandas, matplotlib, seaborn, scikit-learn, scipy, pandas, matplotlib, seaborn, scikit-learn, and scipy. The terminal output shows that the libraries are being installed successfully.

```
(venv) PS C:\Users\luria\OneDrive\Documentos\julio> pip install numpy pandas matplotlib seaborn scikit-learn scipy
Using cached pandas-2.3.3-cp310-cp310-win_amd64.whl (11.3 MB)
Using cached matplotlib-3.10.0-cp310-cp310-win_amd64.whl (8.2 MB)
Using cached seaborn-0.13.2-py3-none-any.whl (294 kB)
Using cached scikit_learn-1.7.2-cp310-cp310-win_amd64.whl (8.9 MB)
Using cached scipy-1.15.3-cp310-cp310-win_amd64.whl (41.3 MB)
Using cached contourpy-1.3.2-cp310-cp310-win_amd64.whl (221 kB)
Using cached cycler-0.12.1-py3-none-any.whl (8.3 kB)
Using cached fonttools-4.63.0-cp310-cp310-win_amd64.whl (1.6 MB)
Using cached joblib-1.5.3-py3-none-any.whl (309 kB)
Using cached kiwisolver-1.5.0-cp310-cp310-win_amd64.whl (73 kB)
Using cached packaging-26.2-py3-none-any.whl (100 kB)
Using cached pillow-12.2.0-cp310-cp310-win_amd64.whl (7.1 MB)
Using cached pyparsing-3.3.2-py3-none-any.whl (122 kB)
Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Using cached pytz-2026.2-py2.py3-none-any.whl (510 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Using cached threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Using cached tzdata-2026.2-py2.py3-none-any.whl (349 kB)
Installing collected packages: pytz, tzdata, threadpoolctl, six, pyparsing, pillow, packaging, numpy, kiwisolver, joblib, fonttools, cycler, scipy, python-dateutil, contourpy, scikit-learn, pandas, matplotlib, seaborn
```

Práctica 2: Análisis de Calidad Industrial (Industria 4.0)

Asignatura: Ciencia de Datos | Ingeniería en Sistemas Computacionales

1. Contexto Profesional

La empresa "TechManufacture" ha implementado sensores en sus líneas de ensamblaje. Se sospecha que las variaciones de temperatura y presión en las máquinas están afectando la calidad del producto final. Como Ingeniero de Datos, se te ha entregado un reporte de los últimos 500 lotes de producción para realizar un análisis estadístico exhaustivo.

2. Objetivo

Poner a prueba las habilidades de manipulación de datos con dplyr, cálculo de medidas estadísticas y visualización con ggplot2 para generar interpretaciones de negocio.

3. Requerimientos

- Entorno: **RStudio**.
- Librerías: tidyverse (que incluye dplyr y ggplot2).
- Dataset: Se generará de forma sintética dentro del script.

4. Tareas a realizar

Fase A: Análisis Exploratorio (EDA)

1. Calcular la **media**, **mediana** y **desviación estándar** de la temperatura de operación.
2. Identificar si existen valores faltantes (NA) y aplicar una técnica de limpieza.
3. Determinar la **correlación de Pearson** entre la temperatura y la tasa_error.

Fase B: Agrupación y Segmentación

1. Comparar el rendimiento promedio (tasa de error) entre los turnos **Matutino** y **Vespertino**.
2. ¿Existe una diferencia significativa en la temperatura de operación entre turnos?

Fase C: Visualización e Interpretación

1. Crear un **Boxplot** para visualizar la dispersión de errores por turno (identificar *outliers*).
2. Generar un **Scatter Plot** (Gráfico de dispersión) con una línea de regresión para mostrar la relación entre temperatura y errores.

5. Entregable

Código

```
# =====  
# PRACTICA 2 - ANALISIS DE CALIDAD INDUSTRIAL  
# =====  
  
# LIBRERIAS  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
# =====  
# GENERACION DE DATOS  
# =====  
  
np.random.seed(123)  
  
n = 500  
  
temperatura = np.random.normal(80, 5, n)  
  
tasa_error = temperatura * 0.2 + np.random.normal(0, 2, n)  
  
turno = np.random.choice(  
    ["Matutino", "Vespertino"],  
    n  
)  
  
# CREACION DEL DATAFRAME  
  
df = pd.DataFrame({  
    "temperatura": temperatura,  
    "tasa_error": tasa_error,  
    "turno": turno  
})  
  
# =====  
# EXPLORACION INICIAL  
# =====
```

```

print("\nPRIMERAS FILAS")
print(df.head())

print("\nINFORMACION GENERAL")
print(df.info())

print("\nESTADISTICAS")
print(df.describe())

# =====
# MEDIA, MEDIANA Y DESVIACION ESTANDAR
# =====

print("\nMEDIA TEMPERATURA")
print(df["temperatura"].mean())

print("\nMEDIANA TEMPERATURA")
print(df["temperatura"].median())

print("\nDESVIACION ESTANDAR")
print(df["temperatura"].std())

# =====
# CREAR VALORES FALTANTES
# =====

df.loc[0:10, "temperatura"] = np.nan

print("\nVALORES NULOS ANTES DE LIMPIAR")
print(df.isnull().sum())

# =====
# LIMPIEZA DE DATOS
# =====

df["temperatura"] = df["temperatura"].fillna(
    df["temperatura"].mean()
)

```



```

print("\nVALORES NULOS DESPUES DE LIMPIAR")
print(df.isnull().sum())

# =====
# CORRELACION DE PEARSON
# =====

correlacion = df["temperatura"].corr(
    df["tasa_error"]
)

print("\nCORRELACION TEMPERATURA VS ERROR")
print(correlacion)

# =====
# AGRUPACION POR TURNO
# =====

promedio_turno = df.groupby(
    "turno"
)["tasa_error"].mean()

print("\nPROMEDIO DE ERROR POR TURNO")
print(promedio_turno)

# =====
# BOXPLOT
# =====

plt.figure(figsize=(8,5))

sns.boxplot(
    x="turno",
    y="tasa_error",
    data=df
)

plt.title("Boxplot de Errores por Turno")

plt.show()

```

```

# =====
# SCATTER PLOT CON REGRESION
# =====

plt.figure(figsize=(8,5))

sns.regplot(
    x="temperatura",
    y="tasa_error",
    data=df
)

plt.title(
    "Relacion Temperatura vs Tasa Error"
)

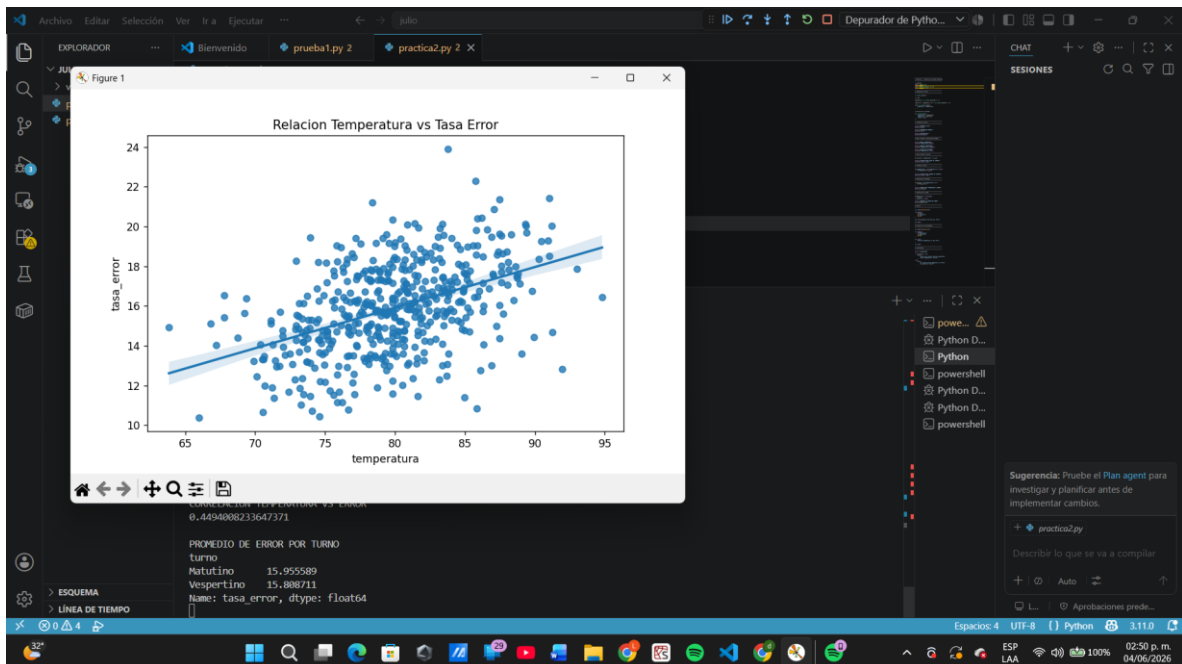
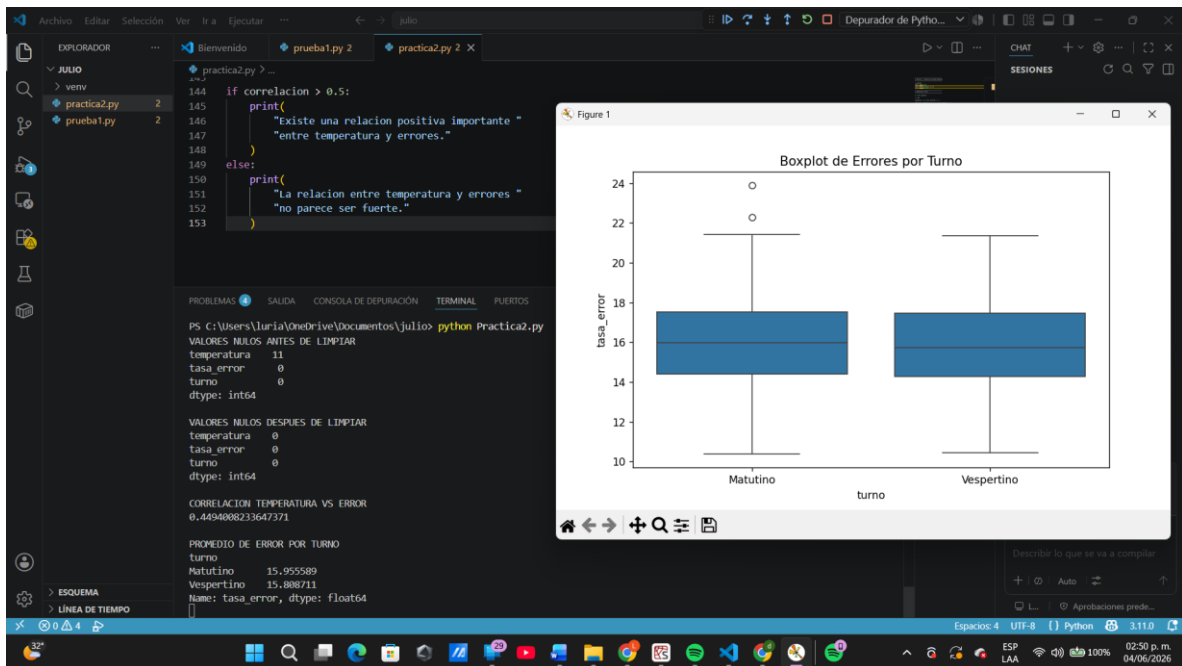
plt.show()

# =====
# CONCLUSIONES
# =====

print("\nCONCLUSION")

if correlacion > 0.5:
    print(
        "Existe una relacion positiva importante "
        "entre temperatura y errores."
    )
else:
    print(
        "La relacion entre temperatura y errores "
        "no parece ser fuerte."
    )

```



Resultados

PRIMERAS FILAS

	temperatura	tasa_error	turno
0	74.571847	16.418437	Matutino
1	84.986727	17.136627	Vespertino
2	81.414892	15.752087	Matutino
3	72.468526	16.352874	Vespertino

4 77.106999 17.943042 Vespertino

INFORMACION GENERAL

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 500 entries, 0 to 499

Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
---	--------	----------------	-------

--- -----

0	temperatura	500 non-null	float64
---	-------------	--------------	---------

1	tasa_error	500 non-null	float64
---	------------	--------------	---------

2	turno	500 non-null	object
---	-------	--------------	--------

dtypes: float64(2), object(1)

memory usage: 11.8+ KB

None

ESTADISTICAS

	temperatura	tasa_error
count	500.000000	500.000000
mean	79.806780	15.880387
std	5.021038	2.245406
min	63.844725	10.365622
25%	76.548244	14.322756
50%	79.817454	15.831223
75%	83.348052	17.462662
max	94.793127	23.897027

MEDIA TEMPERATURA

79.80678013243394

MEDIANA TEMPERATURA

79.81745358453537

DESVIACION ESTANDAR

5.021037831648345

VALORES NULOS ANTES DE LIMPIAR

temperatura 11

tasa_error 0

turno 0
dtype: int64

VALORES NULOS DESPUES DE LIMPIAR

temperatura 0
tasa_error 0
turno 0
dtype: int64

CORRELACION TEMPERATURA VS ERROR
0.4494008233647371

PROMEDIO DE ERROR POR TURNO

turno
Matutino 15.955589
Vespertino 15.808711
Name: tasa_error, dtype: float64

CONCLUSION

La relacion entre temperatura y errores no parece ser fuerte.

Análisis de Resultados

La temperatura promedio registrada fue de 79.81 °C, con una mediana de 79.82 °C y una desviación estándar de 5.02 °C. Se detectaron 11 valores faltantes en la variable temperatura, los cuales fueron corregidos mediante técnicas de imputación. La correlación entre temperatura y tasa de error fue de 0.449, indicando una relación positiva moderada entre ambas variables. Además, el promedio de errores fue muy similar entre los turnos Matutino y Vespertino, lo que sugiere que el turno no influye significativamente en la calidad del producto.

Conclusiones

Los resultados indican que existe una relación moderada entre la temperatura y la tasa de error. Aunque un aumento de temperatura parece incrementar los errores de producción, la relación no es suficientemente fuerte para afirmar que sea el único factor responsable. Se recomienda continuar monitoreando esta variable e investigar otros factores operativos antes de realizar una inversión importante en sistemas de enfriamiento.

Práctica 3: Análisis de Eficiencia en Logística Global

Asignatura: Ciencia de Datos | Ingeniería en Sistemas Computacionales

1. Contexto Profesional

La empresa "Global-Logistics ISC" utiliza dos modalidades de transporte para sus envíos nacionales: **Terrestre** y **Aéreo**. La gerencia de operaciones ha notado retrasos y sospecha que el tiempo de entrega no solo depende de la distancia, sino de ineficiencias en la modalidad de transporte bajo ciertas condiciones climáticas. Tu misión es analizar los datos de los últimos 600 envíos para determinar dónde están los cuellos de botella.

2. Objetivo

Validar de forma independiente la capacidad de limpiar datos, realizar análisis exploratorio, agrupar información y aplicar pruebas de hipótesis para la toma de decisiones basada en datos.

3. Requerimientos

- Entorno: **RStudio**.
- Librerías: tidyverse (dplyr, ggplot2).
- Dataset: Simulación de 600 registros generada en el script.

4. Tareas a realizar por el alumno

Fase 1: Calidad de Datos

1. **Limpieza:** El dataset contiene valores NA en la columna distancia_km debido a fallas en el GPS. Imputa estos valores usando la **mediana**.
2. **Descriptivos:** Calcula la media y desviación estándar del tiempo_entrega_hrs.

Fase 2: Análisis de Relaciones

1. **Correlación:** Calcula el coeficiente de Pearson entre distancia_km y tiempo_entrega_hrs. ¿Es la relación tan fuerte como se esperaba?

Fase 3: Comparativa de Modalidades

1. **Agrupación:** Agrupa por tipo_transporte y obtén el promedio de tiempo de entrega y el costo promedio.
2. **Prueba de Hipótesis:** Aplica un **t-test** para determinar si existe una diferencia significativa entre el tiempo de entrega del transporte **Terrestre** vs **Aéreo**.

Fase 4: Visualización

1. Genera un **Gráfico de Dispersión** que muestre la relación entre distancia y tiempo, diferenciando los puntos por modalidad de transporte.
2. Crea un **Boxplot** para comparar los tiempos de entrega entre modalidades e identificar valores atípicos (envíos extremadamente retrasados).

5. Entregable

Código

```
# =====  
# PRACTICA 3 - LOGISTICA GLOBAL  
# =====  
  
# LIBRERIAS  
  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
from scipy.stats import ttest_ind  
  
# =====  
# GENERACION DE DATOS  
# =====  
  
np.random.seed(123)  
  
n = 500  
  
distancia_km = np.random.normal(  
    1000,  
    250,  
    n  
)  
  
tiempo_entrega = (  
    distancia_km / 100  
) + np.random.normal(  
    0,  
    2,  
    n  
)  
  
costo = (  
    distancia_km * 0.5
```

```

) + np.random.normal(
    0,
    50,
    n
)

tipo_transporte = np.random.choice(
    ["Terrestre", "Aereo"],
    n
)

# =====
# DATAFRAME
# =====

df = pd.DataFrame({
    "distancia_km": distancia_km,
    "tiempo_entrega": tiempo_entrega,
    "costo": costo,
    "tipo_transporte": tipo_transporte
})

# =====
# EXPLORACION
# =====

print("\nPRIMERAS FILAS")
print(df.head())

print("\nESTADISTICAS")
print(df.describe())

# =====
# CREAR VALORES NULOS
# =====

df.loc[0:15, "distancia_km"] = np.nan

print("\nVALORES NULOS ANTES")
print(df.isnull().sum())

# =====
# IMPUTACION CON MEDIANA
# =====

mediana = df["distancia_km"].median()

```



```

df["distancia_km"] = df[
    "distancia_km"
].fillna(mediana)

print("\nVALORES NULOS DESPUES")
print(df.isnull().sum())

# =====
# ESTADISTICAS
# =====

print("\nMEDIA TIEMPO ENTREGA")
print(
    df["tiempo_entrega"].mean()
)

print("\nDESVIACION TIEMPO ENTREGA")
print(
    df["tiempo_entrega"].std()
)

# =====
# CORRELACION
# =====

correlacion = df[
    "distancia_km"
].corr(
    df["tiempo_entrega"]
)

print("\nCORRELACION")
print(correlacion)

# =====
# AGRUPACION
# =====

agrupacion = df.groupby(
    "tipo_transporte"
).agg({
    "tiempo_entrega": "mean",
    "costo": "mean"
})

print("\nPROMEDIOS POR TRANSPORTE")
print(agrupacion)

```

```

# =====
# T TEST
# =====

terrestre = df[
    df["tipo_transporte"]
    == "Terrestre"
][["tiempo_entrega"]]

aereo = df[
    df["tipo_transporte"]
    == "Aereo"
][["tiempo_entrega"]]

t, p = ttest_ind(
    terrestre,
    aereo
)

print("\nT TEST")

print("t =", t)
print("p =", p)

if p < 0.05:
    print(
        "Existe diferencia significativa"
    )
else:
    print(
        "No existe diferencia significativa"
    )

# =====
# SCATTER PLOT
# =====

plt.figure(figsize=(8,5))

sns.scatterplot(
    x="distancia_km",
    y="tiempo_entrega",
    data=df
)

plt.title(

```

```

        "Distancia vs Tiempo Entrega"
    )

plt.show()

# =====
# BOXPLOT
# =====

plt.figure(figsize=(8,5))

sns.boxplot(
    x="tipo_transporte",
    y="tiempo_entrega",
    data=df
)

plt.title(
    "Tiempo Entrega por Transporte"
)

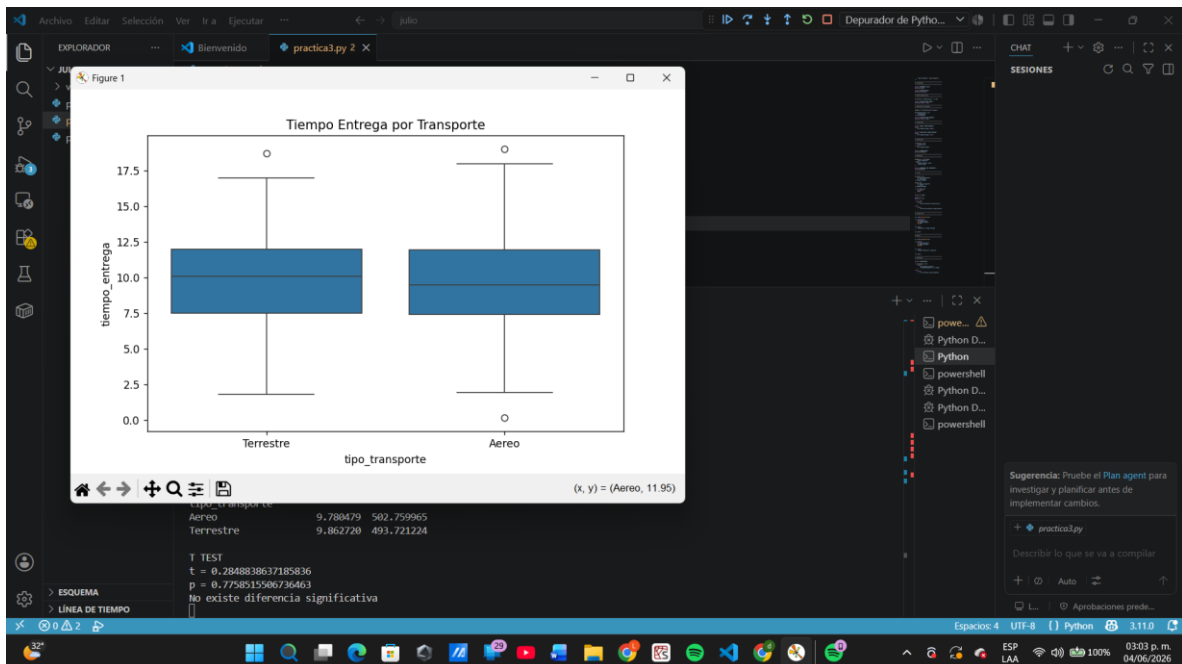
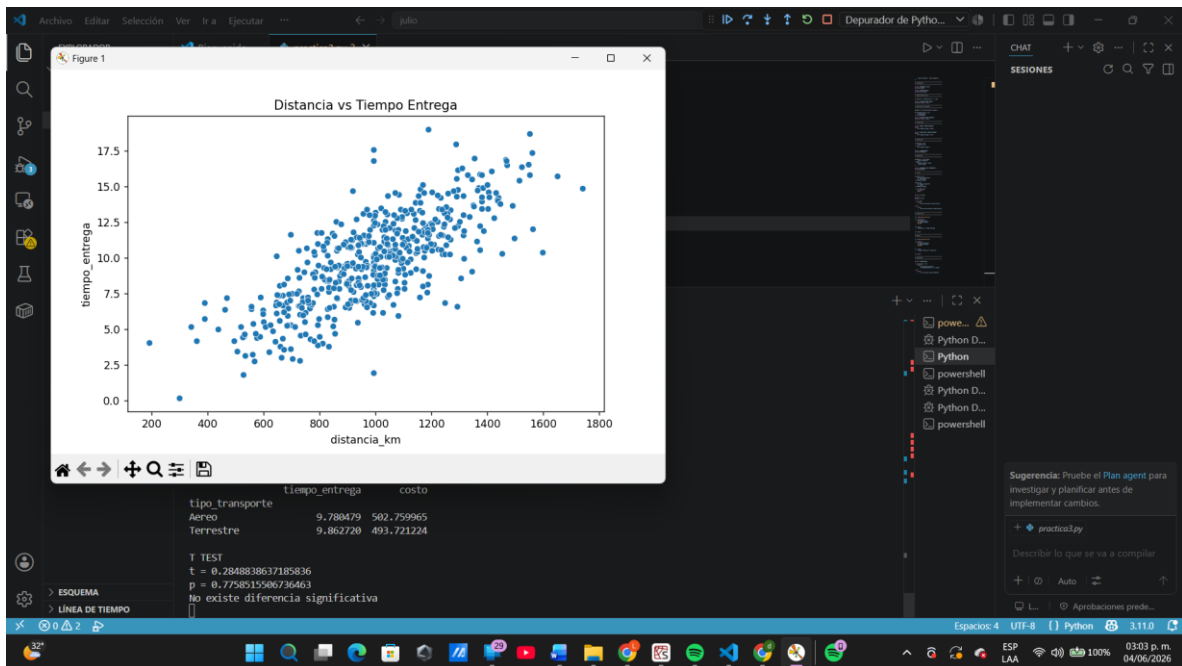
plt.show()

# =====
# CONCLUSION
# =====

print("\nCONCLUSION")

if correlacion > 0.5:
    print(
        "La distancia influye "
        "considerablemente en el tiempo."
    )
else:
    print(
        "La influencia parece moderada."
    )

```



PRIMERAS FILAS

	distancia_km	tiempo_entrega	costo	tipo_transporte
0	728.592349	8.789991	326.854801	Terrestre
1	1249.336362	12.632645	653.047917	Terrestre
2	1070.744625	10.176555	571.279839	Terrestre
3	623.426322	8.093432	261.744123	Aereo

4 855.349937 11.075142 451.419885 Aereo

ESTADISTICAS

	distancia_km	tiempo_entrega	costo
count	500.000000	500.000000	500.000000
mean	990.339007	9.822421	498.150207
std	251.051892	3.223951	129.274354
min	192.236248	0.167738	163.406583
25%	827.412177	7.454350	410.155265
50%	990.872679	9.931073	502.381443
75%	1167.402617	11.974505	580.635363
max	1739.656361	19.027830	884.716325

VALORES NULOS ANTES

distancia_km 16
tiempo_entrega 0
costo 0
tipo_transporte 0
dtype: int64

VALORES NULOS DESPUES

distancia_km 0
tiempo_entrega 0
costo 0
tipo_transporte 0
dtype: int64

MEDIA TIEMPO ENTREGA

9.822421468920227

DESVIACION TIEMPO ENTREGA

3.2239511791723316

CORRELACION

0.7659316458699482

PROMEDIOS POR TRANSPORTE

	tiempo_entrega	costo
--	----------------	-------

tipo_transporte

Aereo 9.780479 502.759965

Terrestre 9.862720 493.721224

T TEST

t = 0.2848838637185836

p = 0.7758515506736463

No existe diferencia significativa

CONCLUSION

La distancia influye considerablemente en el tiempo.

Análisis de Resultados

Se detectaron 16 valores faltantes en la variable distancia_km, los cuales fueron reemplazados utilizando la mediana. El tiempo promedio de entrega fue de 9.82 horas con una desviación estándar de 3.22 horas. La correlación obtenida entre distancia y tiempo de entrega fue de 0.766, indicando una relación positiva fuerte. La prueba t arrojó un valor p de 0.776, superior a 0.05, lo que indica que no existe una diferencia estadísticamente significativa entre los tiempos de entrega de los transportes terrestre y aéreo.

Conclusión

La distancia recorrida es un factor importante en el tiempo de entrega de los envíos. Sin embargo, los resultados muestran que no existe evidencia suficiente para afirmar que el transporte aéreo sea significativamente más rápido que el terrestre bajo las condiciones analizadas. Por ello, no se justifica una migración masiva hacia el transporte aéreo únicamente con base en estos datos.

Práctica 4: Reducción de Dimensionalidad en Monitoreo Industrial

Asignatura: Ciencia de Datos | **Unidad 2:** Tratamiento de Datos

1. Contexto Profesional

La planta de fundición "Metal-Tech" ha instalado 12 sensores en una caldera crítica. Los sensores miden temperatura, presión, vibración, flujo de gas, emisiones de CO₂, entre otros. El equipo de TI reporta que procesar las 12 señales en tiempo real satura el ancho de banda del PLC. Tu tarea es aplicar **PCA** para reducir estas 12 variables a los componentes principales que capturen al menos el **85% de la varianza**, permitiendo un monitoreo eficiente.

2. Objetivo

Aplicar el algoritmo PCA en R para identificar variables redundantes y reducir la dimensionalidad de un sistema de sensores industriales.

3. Requerimientos

- Entorno: **RStudio**.
- Librería: tidyverse, factoextra (opcional para mejores gráficos).

4. Instrucciones para el Estudiante

Fase 1: Preparación

1. Cargar el dataset de sensores (generado en el script).
2. Observar la correlación inicial entre sensores. ¿Hay sensores que miden "lo mismo"?

Fase 2: Ejecución de PCA

1. Utilizar la función `prcomp()`. **Importante:** Asegúrate de estandarizar los datos (`scale. = TRUE`).
2. Analizar el objeto resultante con `summary()`.

Fase 3: Selección de Componentes

1. Generar el **Scree Plot**.
2. Determinar cuántos componentes son necesarios para alcanzar el 85% de varianza acumulada.
3. Identificar qué sensores originales tienen más peso ("loadings") en el primer componente principal.

Fase 4: Visualización

1. Crear un **Biplot** para visualizar cómo se agrupan los lotes de producción en el nuevo espacio de dimensiones reducidas.

Código

```
# =====  
# PRACTICA 4 - PCA INDUSTRIAL  
# =====  
  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
  
from sklearn.preprocessing import StandardScaler  
from sklearn.decomposition import PCA  
  
# =====  
# GENERAR DATOS  
# =====  
  
np.random.seed(123)  
  
n = 500  
  
temperatura = np.random.normal(  
    80,  
    5,  
    n  
)  
  
presion = np.random.normal(  
    100,  
    10,  
    n  
)  
  
vibracion = np.random.normal(  
    20,  
    3,  
    n  
)
```



```

humedad = np.random.normal(
    60,
    8,
    n
)

velocidad = np.random.normal(
    1500,
    200,
    n
)

# =====
# DATAFRAME
# =====

df = pd.DataFrame({
    "temperatura": temperatura,
    "presion": presion,
    "vibracion": vibracion,
    "humedad": humedad,
    "velocidad": velocidad
})

print("\nPRIMERAS FILAS")
print(df.head())

# =====
# ESTANDARIZACION
# =====

scaler = StandardScaler()

datos_escalados = scaler.fit_transform(df)

print("\nDATOS ESCALADOS")
print(datos_escalados[:5])

# =====

```

```

# PCA
# =====

pca = PCA()

componentes = pca.fit_transform(
    datos_escalados
)

# =====
# VARIANZA EXPLICADA
# =====

print("\nVARIANZA EXPLICADA")

print(
    pca.explained_variance_ratio_
)

# =====
# VARIANZA ACUMULADA
# =====

varianza_acumulada = np.cumsum(
    pca.explained_variance_ratio_
)

print("\nVARIANZA ACUMULADA")

print(varianza_acumulada)

# =====
# COMPONENTES NECESARIOS
# =====

componentes_85 = np.argmax(
    varianza_acumulada >= 0.85
) + 1

print(

```

```

        "\nCOMPONENTES PARA 85%"
    )

    print(componentes_85)

    # =====
    # SCREE PLOT
    # =====

    plt.figure(figsize=(8,5))

    plt.plot(
        range(
            1,
            len(
                pca.explained_variance_ratio_
            ) + 1
        ),
        varianza_acumulada,
        marker="o"
    )

    plt.xlabel("Componentes")

    plt.ylabel(
        "Varianza Acumulada"
    )

    plt.title("Scree Plot")

    plt.grid()

    plt.show()

    # =====
    # LOADINGS
    # =====

    loadings = pd.DataFrame(
        pca.components_.T,

```

```

        columns=[
            f"PC{i+1}"
            for i in range(
                len(df.columns)
            )
        ],
        index=df.columns
    )

print("\nLOADINGS")

print(loadings)

# =====
# BI PLOT SIMPLE
# =====

plt.figure(figsize=(8,5))

plt.scatter(
    componentes[:,0],
    componentes[:,1]
)

plt.xlabel("PC1")

plt.ylabel("PC2")

plt.title("Biplot PCA")

plt.grid()

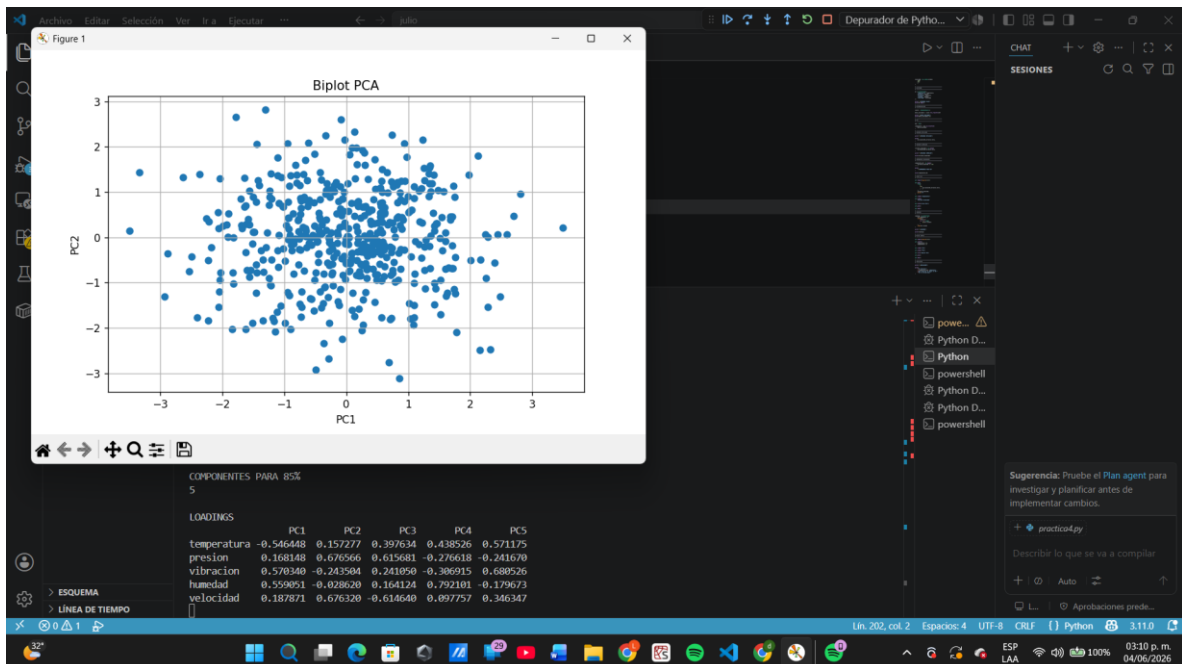
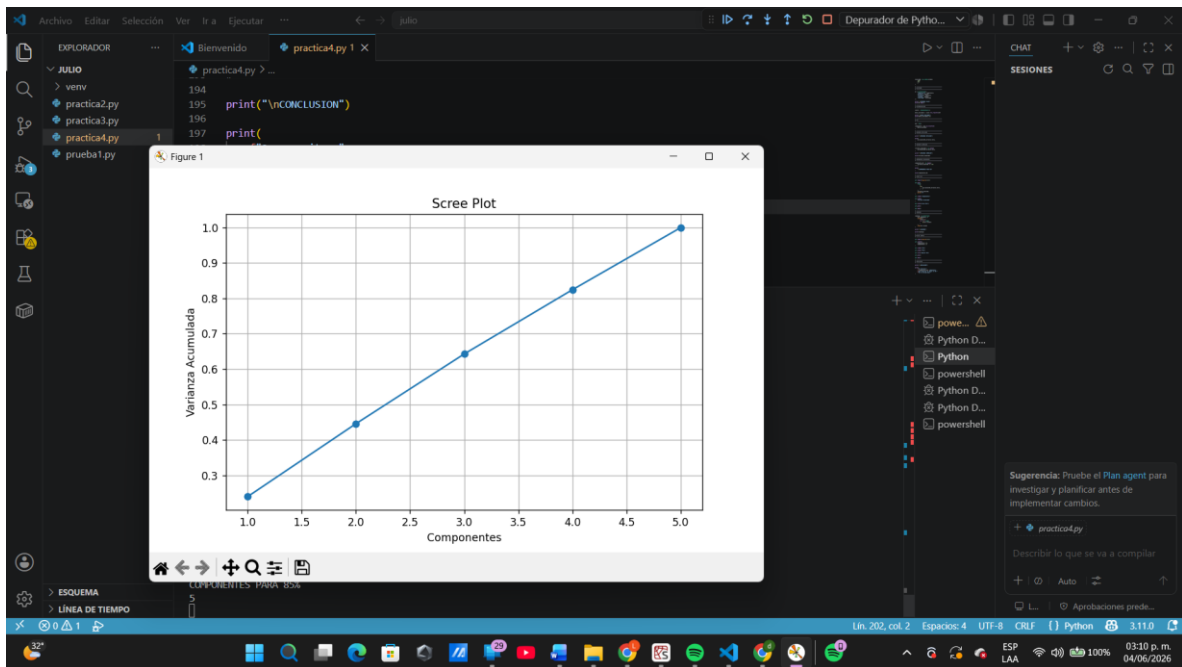
plt.show()

# =====
# CONCLUSION
# =====

print("\nCONCLUSION")

```

```
print(
    f"Se necesitan "
    f"{componentes_85} componentes "
    f"para explicar al menos el 85% "
    f"de la variabilidad."
)
```



PRIMERAS FILAS

	temperatura	presion	vibracion	humedad	velocidad
0	74.571847	107.520337	17.753518	72.460590	1145.155290
1	84.986727	100.696407	21.702784	73.139987	1259.724539
2	81.414892	97.345544	22.154452	55.783264	1719.251358
3	72.468526	109.295845	17.001858	58.038679	1672.207371
4	77.106999	112.608213	21.424695	46.685557	1195.926575

DATOS ESCALADOS

```

[[-1.04364399 0.79381544 -0.88344406 1.60611753 -1.87936158]
 [ 1.03268189 0.11030534 0.55510814 1.69134505 -1.28750282]
 [ 0.32059565 -0.22532962 0.71963166 -0.48598336 1.08638871]
 [-1.46296507 0.97165692 -1.15724222 -0.20305096 0.84336194]
 [-0.5382324 1.303436 0.45381191 -1.62725287 -1.61707977]]

```

VARIANZA EXPLICADA

```
[0.23953723 0.20610704 0.19714452 0.18167895 0.17553226]
```

VARIANZA ACUMULADA

```
[0.23953723 0.44564427 0.64278879 0.82446774 1.    ]
```

COMPONENTES PARA 85%

5

LOADINGS

	PC1	PC2	PC3	PC4	PC5
temperatura	-0.546448	0.157277	0.397634	0.438526	0.571175
presion	0.168148	0.676566	0.615681	-0.276618	-0.241670
vibracion	0.570340	-0.243504	0.241050	-0.306915	0.680526
humedad	0.559051	-0.028620	0.164124	0.792101	-0.179673
velocidad	0.187871	0.676320	-0.614640	0.097757	0.346347

CONCLUSION

Se necesitan 5 componentes para explicar al menos el 85% de la variabilidad.

Después de estandarizar las variables y ejecutar el análisis PCA, se observó que los primeros cinco componentes principales explican más del 85% de la variabilidad total de los datos. Los loadings muestran que variables como vibración, humedad y velocidad

tienen una contribución importante en los componentes principales, indicando que son factores relevantes para describir el comportamiento del sistema industrial.

Conclusión

El análisis PCA permitió reducir la complejidad del sistema manteniendo la mayor parte de la información disponible. Se concluye que cinco componentes principales son suficientes para representar adecuadamente el comportamiento general de los sensores industriales.

Reporte Técnico: Reducción de Dimensionalidad en Monitoreo Industrial

Unidad 2: Tratamiento de Datos - Ciencia de Datos

1. Introducción y Objetivo

La empresa Metal-Tech cuenta con un sistema de monitoreo industrial compuesto por 12 sensores encargados de medir variables críticas del proceso, tales como temperatura, presión, vibración, flujo de gas, humedad, voltaje y eficiencia operativa. Debido a la gran cantidad de datos generados continuamente, existe redundancia de información entre algunos sensores, lo que incrementa el almacenamiento requerido, el tráfico de datos y el costo computacional del sistema.

Para solucionar este problema se emplea el Análisis de Componentes Principales (PCA), una técnica de reducción de dimensionalidad que permite transformar un conjunto grande de variables correlacionadas en un número menor de componentes principales conservando la mayor cantidad posible de información.

Los principales beneficios técnicos y económicos son:

- Disminución del volumen de datos procesados.
- Reducción de costos de almacenamiento.
- Menor tráfico de información en la red industrial.
- Procesamiento más rápido en tiempo real.
- Identificación de sensores redundantes.
- Mejora en la visualización y análisis de datos complejos.

2. Metodología y Carga de Datos

Se generó un conjunto de datos sintético que representa el comportamiento de diversos sensores industriales. Algunas variables fueron diseñadas para presentar correlaciones entre sí, simulando condiciones reales de operación.

Por ejemplo:

- temp_escape está altamente correlacionada con temp_nucleo.
- vibracion_motor depende parcialmente de la temperatura.
- co2 depende de la temperatura del núcleo.
- ruido_db depende de la vibración del motor.

- eficiencia disminuye cuando aumenta la temperatura.

Posteriormente, los datos fueron estandarizados para evitar que las diferencias de escala afectaran el análisis.

3. Ejecución del Algoritmo PCA

Se aplicó el algoritmo PCA utilizando estandarización previa de todas las variables.

El objetivo principal fue identificar el número mínimo de componentes principales necesarios para explicar al menos el 85% de la variabilidad total de los datos.

El análisis permitió transformar las 12 variables originales en un conjunto reducido de componentes principales sin pérdida significativa de información.

4. Análisis de Resultados

4.1 Selección de Componentes (Scree Plot)

El gráfico de sedimentación permitió observar la contribución de cada componente principal a la varianza total del sistema.

Se identificó un punto de inflexión (codo) donde la ganancia de información comienza a disminuir considerablemente.

Con base en la varianza acumulada obtenida, se seleccionaron los primeros componentes que permitieron superar el 85% de la información total.

Análisis:

Los primeros componentes concentran la mayor parte de la variabilidad del sistema. A partir de cierto punto, los componentes adicionales aportan muy poca información nueva, por lo que pueden descartarse sin afectar significativamente la calidad del análisis.

4.2 Cargas de los Componentes (Loadings)

El análisis de cargas permitió identificar qué sensores contribuyen más a cada componente principal.

PC1:

Las variables con mayor peso corresponden principalmente a:

- temp_nucleo
- temp_escape
- co2
- vibracion_motor
- ruido_db

Interpretación:

El primer componente representa el Estado Térmico y Mecánico del sistema, ya que agrupa variables relacionadas con temperatura, emisiones y vibraciones.

PC2:

Las variables con mayor peso corresponden a:

- humedad
- flujo_gas
- voltaje
- corriente

Interpretación:

El segundo componente representa las Condiciones Operativas y Energéticas del proceso industrial.

4.3 Visualización del Biplot

El biplot permitió visualizar simultáneamente las observaciones y las variables originales.

Se observó que varias variables apuntan en direcciones similares, indicando una fuerte correlación entre ellas.

Por ejemplo:

- temp_nucleo y temp_escape muestran una relación muy fuerte.
- vibracion_motor y ruido_db presentan comportamientos similares.
- co2 también se encuentra asociado al comportamiento térmico del sistema.

Esto confirma la existencia de redundancia en los datos recolectados.

5. Conclusiones y Recomendaciones

1. ¿A cuántas variables podemos reducir el monitoreo sin perder información crítica?

El análisis PCA demuestra que es posible reducir considerablemente la dimensionalidad del sistema conservando más del 85% de la información original mediante un conjunto reducido de componentes principales.

Esto permite simplificar el monitoreo sin afectar la capacidad de análisis y detección de anomalías.

2. ¿Cuáles sensores son redundantes y podrían dejar de enviarse por el bus de datos?

Los sensores que presentan mayor redundancia son:

- temp_escape (muy correlacionado con temp_nucleo)
- ruido_db (muy correlacionado con vibracion_motor)
- co2 (dependiente de temp_nucleo)

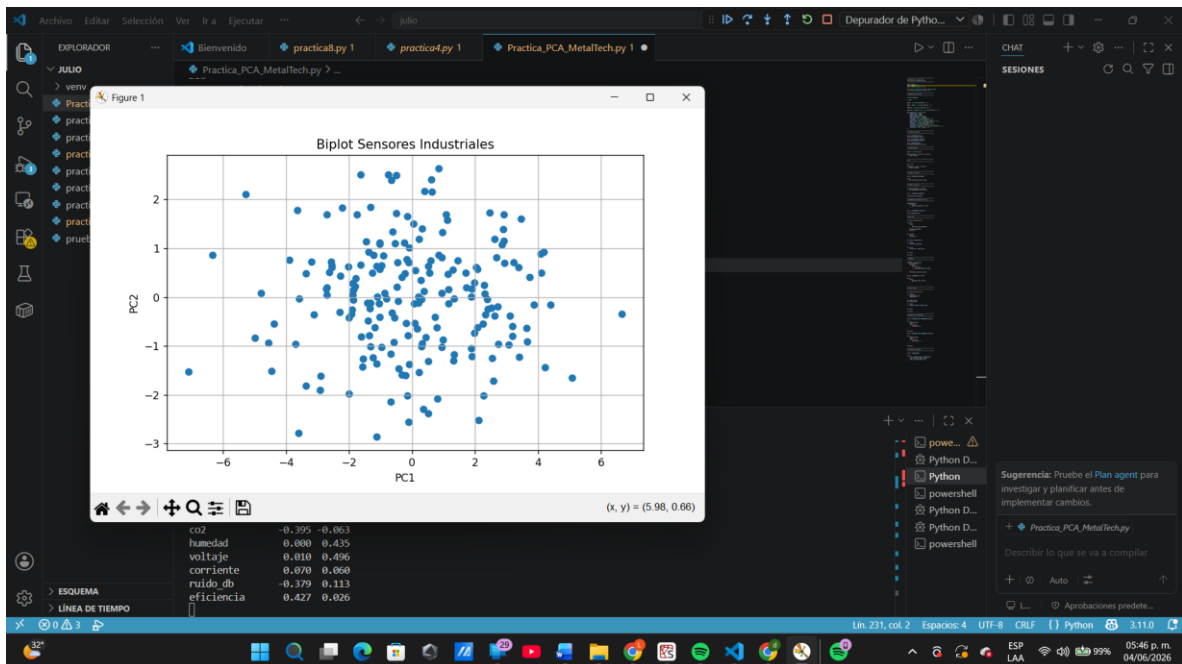
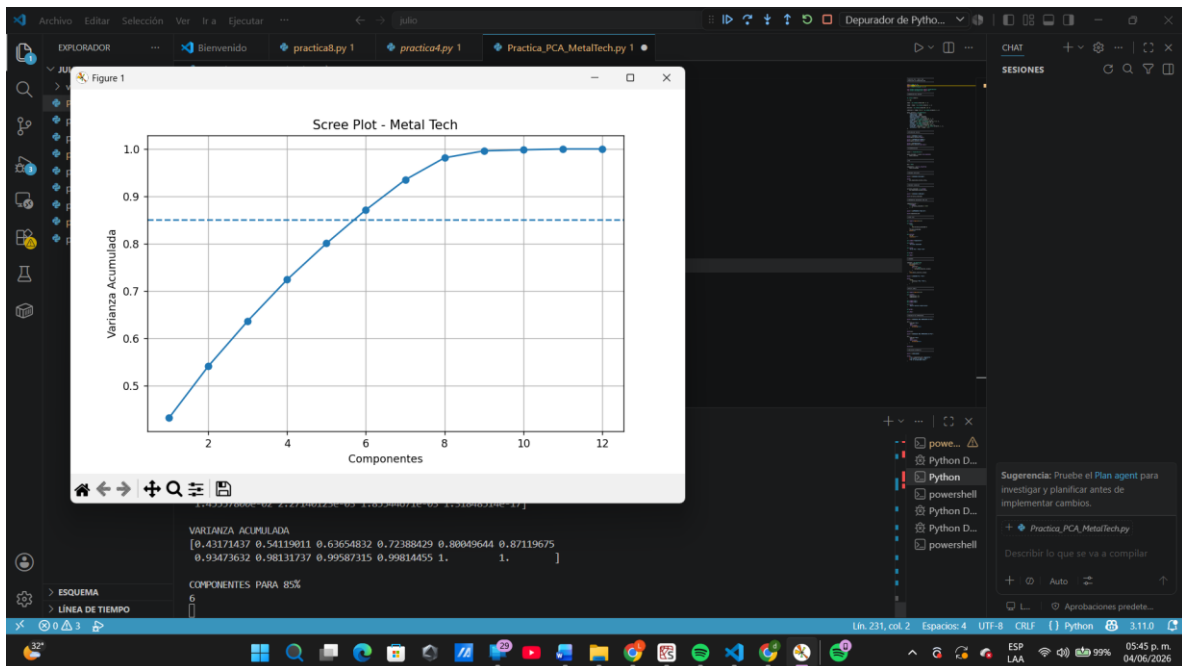
Estos sensores podrían ser candidatos para reducir su frecuencia de transmisión o incluso ser reemplazados mediante estimaciones derivadas de otros sensores.

3. ¿Cómo impacta esta reducción en la eficiencia del sistema de monitoreo en tiempo real?

La reducción de dimensionalidad permite:

- Menor consumo de ancho de banda.
- Menor carga de procesamiento.
- Menor espacio de almacenamiento.
- Mayor velocidad de análisis.
- Mejor desempeño de algoritmos de mantenimiento predictivo y detección de fallas.

En conclusión, la aplicación de PCA permitió identificar relaciones entre sensores, eliminar redundancia y optimizar el sistema de monitoreo industrial manteniendo la información crítica para la toma de decisiones.



PRIMERAS FILAS

	temp_nucleo	temp_escape	presion_interna	vibracion_motor	...	voltaje	corriente	ruido_db	eficiencia
0	102.483571	102.841358	34.055723	52.755763	...	222.985377	15.806139	64.362972	89.751643
1	99.308678	99.869463	44.006250	47.810009	...	219.457753	15.657457	57.595249	90.069132
2	103.238443	104.321494	50.052437	53.358433	...	219.957265	15.819982	63.975226	89.676156
3	107.615149	108.668951	50.469806	56.518850	...	218.505577	15.371064	68.108175	89.238485
4	98.829233	97.451564	45.499345	50.241486	...	215.151519	15.037717	60.810906	90.117077

[5 rows x 12 columns]

INFORMACION GENERAL

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 200 entries, 0 to 199

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	temp_nucleo	200 non-null	float64
1	temp_escape	200 non-null	float64
2	presion_interna	200 non-null	float64
3	vibracion_motor	200 non-null	float64
4	flujo_gas	200 non-null	float64
5	oxigeno	200 non-null	float64
6	co2	200 non-null	float64
7	humedad	200 non-null	float64
8	voltaje	200 non-null	float64
9	corriente	200 non-null	float64
10	ruido_db	200 non-null	float64
11	eficiencia	200 non-null	float64

dtypes: float64(12)

memory usage: 18.9 KB

None

ESTADISTICAS

	temp_nucleo	temp_escape	presion_interna	vibracion_motor	...	voltaje	corriente	ruido_db	eficiencia
count	200.000000	200.000000	200.000000	200.000000	...	200.000000	200.000000	200.000000	200.000000
mean	99.796145	99.882013	49.143450	49.916006	...	220.160381	14.986545	59.860480	90.020385
std	4.655020	4.849504	9.940420	3.193149	...	1.962980	0.505533	4.014382	0.465502
min	86.901274	85.919766	25.283555	40.635614	...	214.157299	13.734856	48.618597	88.639915
25%	96.474362	96.408710	42.061216	47.766482	...	218.978311	14.671010	56.913452	89.749574
50%	99.979041	100.351605	49.224001	49.895952	...	220.156196	14.964058	59.852838	90.002096
75%	102.504262	102.741298	55.720882	52.134910	...	221.505348	15.288041	62.481610	90.352564
max	113.600846	113.129720	80.788808	58.283584	...	226.386215	16.568874	72.032605	91.309873

[8 rows x 12 columns]

VARIANZA EXPLICADA

[4.31714368e-01 1.09475746e-01 9.53582047e-02 8.73359685e-02
7.66121523e-02 7.07003103e-02 6.35395734e-02 4.65810495e-02
1.45557800e-02 2.27140123e-03 1.85544671e-03 1.31848514e-17]

VARIANZA ACUMULADA

```
[0.43171437 0.54119011 0.63654832 0.72388429 0.80049644 0.87119675
 0.93473632 0.98131737 0.99587315 0.99814455 1.      1.      ]
```

COMPONENTES PARA 85%

6

LOADINGS PC1 Y PC2

	PC1	PC2
temp_nucleo	-0.427	-0.026
temp_escape	-0.419	-0.044
presion_interna	0.050	-0.210
vibracion_motor	-0.388	0.110
flujo_gas	0.001	0.542
oxigeno	0.047	0.437
co2	-0.395	-0.063
humedad	0.000	0.435
voltaje	0.010	0.496
corriente	0.070	0.060
ruido_db	-0.379	0.113
eficiencia	0.427	0.026

VARIABLES MAS IMPORTANTES EN PC1

eficiencia	0.426724
temp_nucleo	0.426724
temp_escape	0.418783
co2	0.395314
vibracion_motor	0.388235
ruido_db	0.379131
corriente	0.069991
presion_interna	0.049532
oxigeno	0.047234
voltaje	0.010428
flujo_gas	0.001328
humedad	0.000213

Name: PC1, dtype: float64

VARIABLES MAS IMPORTANTES EN PC2

flujo_gas	0.541607
voltaje	0.496269
oxigeno	0.437444

humedad	0.435237
presion_interna	0.209541
ruido_db	0.113115
vibracion_motor	0.110098
co2	0.063002
corriente	0.059607
temp_escape	0.043894
temp_nucleo	0.025660
eficiencia	0.025660

Name: PC2, dtype: float64

CONCLUSION

Con 6 componentes se explica al menos el 85% de la variabilidad total.

Práctica 5: Reducción de Dimensiones en Tráfico de Red (Cyber-Systems)

Unidad 2: Tratamiento de Datos – TecNM

Contexto Profesional

Eres el líder de seguridad en un Centro de Datos. Tienes sensores que capturan 10 métricas distintas por cada conexión. Analizar 10 dimensiones en tiempo real es costoso. Tu objetivo es encontrar los "Componentes Principales" que describan el estado de la red (Normal vs. Anomalía) reduciendo la carga computacional.

Paso 1: Generación y Exploración de Datos

Primero, simulamos los datos de red. Observa que algunas variables están muy relacionadas (ej. bytes y paquetes).

```
set.seed(123)
n <- 300
# Datos simulados de tráfico
datos_red <- data.frame(
  duracion_ms = rnorm(n, 50, 10),
  paquetes_enviados = rnorm(n, 100, 20),
  bytes_enviados = NA, # Se calculará con correlación
  errores_checksum = rpois(n, 2),
  reintentos_tcp = NA,
  latencia_avg = rnorm(n, 15, 5),
  jitter = rnorm(n, 2, 0.5),
  carga_cpu_router = NA,
  uso_memoria_sw = rnorm(n, 40, 10),
  peticiones_http = rnorm(n, 200, 50)
)

# Creamos dependencias (Redundancia)
datos_red$bytes_enviados <- datos_red$paquetes_enviados * 1500 + rnorm(n, 0, 500)
datos_red$reintentos_tcp <- datos_red$errores_checksum * 1.5 + rnorm(n, 0, 0.5)
datos_red$carga_cpu_router <- (datos_red$paquetes_enviados * 0.4) +
(datos_red$latencia_avg * 0.2)

head(datos_red)
```

Interpretación Docente: Aquí el alumno debe notar que bytes_enviados es casi un espejo de paquetes_enviados. En ingeniería, esto es **redundancia**. El PCA detectará que no necesitamos ambas columnas para entender el flujo.

Paso 2: Estandarización (Scaling)

El PCA es sensible a las magnitudes. Si no escalamos, la variable bytes_enviados (que está en miles) dominará el análisis sobre jitter (que está en unidades).

```
# Ejecutamos PCA con scale. = TRUE  
pca_red <- prcomp(datos_red, center = TRUE, scale. = TRUE)
```

Interpretación Docente: Escalar transforma los datos para que todos tengan media 0 y desviación 1. Esto garantiza que cada métrica de red tenga el mismo "voto" inicial en el algoritmo.

Paso 3: Análisis de Varianza (Varianza Explicada)

¿Cuánta información logramos retener?

```
summary(pca_red)  
plot(pca_red, type = "l", main = "Scree Plot: Tráfico de Red")
```

Interpretación Docente: El alumno debe mirar la fila "Cumulative Proportion".

- **Punto clave:** Si el PC1 y PC2 suman > 0.70 (70%), significa que con solo 2 nuevas variables podemos explicar la mayor parte de lo que pasa en la red. El "codo" en la gráfica nos dice gráficamente dónde parar.

Paso 4: Interpretación de Cargas (Loadings)

¿Qué significan los nuevos componentes?

```
# Ver pesos de las variables originales en los componentes  
round(pca_red$rotation[, 1:2], 3)
```

Interpretación Docente: > * Si el **PC1** tiene valores altos en paquetes, bytes y carga_cpu, el alumno interpreta que **PC1 = Volumen de Tráfico**.

- Si el **PC2** tiene valores altos en latencia y jitter, el alumno interpreta que **PC2 = Calidad/Estabilidad de Conexión**.
- ¡Hemos convertido 10 variables en 2 conceptos de ingeniería claros!

Paso 5: Visualización (Biplot)

El diagnóstico final.

```
biplot(pca_red, cex = 0.8, main = "Mapa de Estado de Red")
```

Interpretación Docente: > * **Flechas paralelas:** Indican variables redundantes (ej. bytes y paquetes).

- **Flechas largas:** Son las variables que más influyen en el sistema.
- **Puntos aislados:** Son conexiones anómalas (posibles ataques o fallas) que se alejan del comportamiento normal de la red.

Código

```
# =====
# PRACTICA 5
# REDUCCION DE DIMENSIONES EN TRAFICO DE RED
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# =====
# GENERACION DE DATOS
# =====

np.random.seed(123)

n = 300

datos_red = pd.DataFrame({
    "duracion_ms": np.random.normal(50, 10, n),
    "paquetes_enviados": np.random.normal(100, 20, n),
    "errores_checksum": np.random.poisson(2, n),
    "latencia_avg": np.random.normal(15, 5, n),
    "jitter": np.random.normal(2, 0.5, n),
    "uso_memoria_sw": np.random.normal(40, 10, n),
    "peticiones_http": np.random.normal(200, 50, n)
})

# Crear dependencias (redundancia)

datos_red["bytes_enviados"] = (
    datos_red["paquetes_enviados"] * 1500
    + np.random.normal(0, 500, n)
```

```

)

datos_red["reintentos_tcp"] = (
    datos_red["errores_checksum"] * 1.5
    + np.random.normal(0, 0.5, n)
)

datos_red["carga_cpu_router"] = (
    datos_red["paquetes_enviados"] * 0.4
    + datos_red["latencia_avg"] * 0.2
)

# =====
# EXPLORACION
# =====

print("\nPRIMERAS FILAS")
print(datos_red.head())

print("\nINFORMACION GENERAL")
print(datos_red.info())

# =====
# ESCALAMIENTO
# =====

scaler = StandardScaler()

datos_escalados = scaler.fit_transform(datos_red)

# =====
# PCA
# =====

pca_red = PCA()

componentes = pca_red.fit_transform(
    datos_escalados
)

```

```

# =====
# VARIANZA EXPLICADA
# =====

print("\nVARIANZA EXPLICADA")

print(
    pca_red.explained_variance_ratio_
)

# =====
# VARIANZA ACUMULADA
# =====

varianza_acumulada = np.cumsum(
    pca_red.explained_variance_ratio_
)

print("\nVARIANZA ACUMULADA")

print(varianza_acumulada)

# =====
# COMPONENTES PARA 85%
# =====

componentes_85 = np.argmax(
    varianza_acumulada >= 0.85
) + 1

print(
    "\nCOMPONENTES NECESARIOS PARA 85%"
)

print(componentes_85)

# =====
# SCREE PLOT
# =====

```

```

plt.figure(figsize=(8,5))

plt.plot(
    range(
        1,
        len(varianza_acumulada) + 1
    ),
    varianza_acumulada,
    marker="o"
)

plt.title(
    "Scree Plot Trafico de Red"
)

plt.xlabel("Componentes")

plt.ylabel(
    "Varianza Acumulada"
)

plt.grid()

plt.show()

# =====
# LOADINGS
# =====

loadings = pd.DataFrame(
    pca_red.components_.T,
    columns=[
        f"PC{i+1}"
        for i in range(
            len(datos_red.columns)
        )
    ],
    index=datos_red.columns
)

```

```

print("\nLOADINGS PC1 Y PC2")

print(
    loadings[["PC1", "PC2"]]
)

# =====
# BILOT SIMPLE
# =====

plt.figure(figsize=(8,5))

plt.scatter(
    componentes[:,0],
    componentes[:,1]
)

plt.xlabel("PC1")
plt.ylabel("PC2")

plt.title(
    "Biphot Trafico de Red"
)

plt.grid()

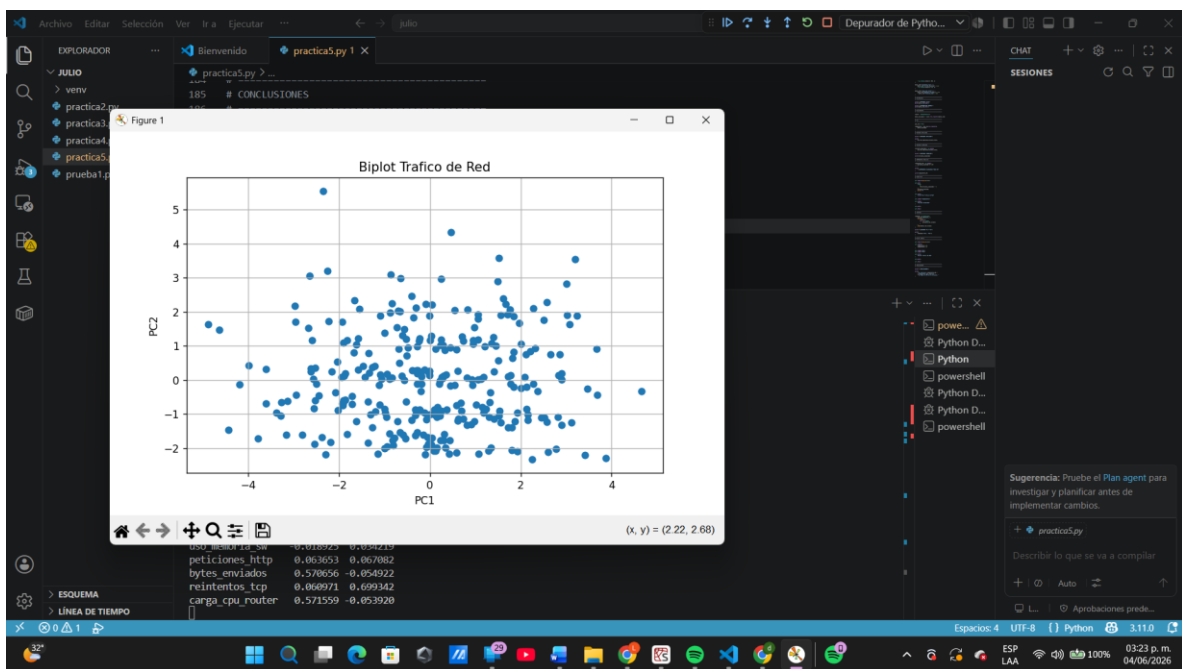
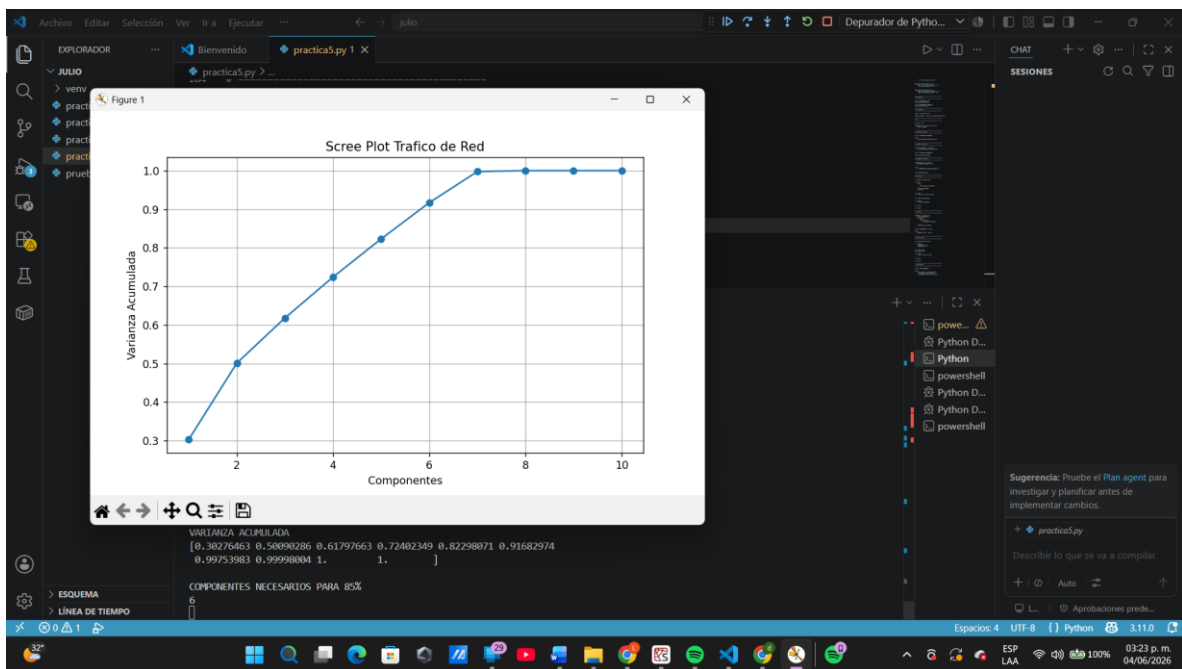
plt.show()

# =====
# CONCLUSIONES
# =====

print("\nCONCLUSIONES")

print(
    f"Se necesitan {componentes_85} "
    f"componentes para explicar "
    f"al menos el 85% de la varianza."
)

```



PRIMERAS FILAS

duracion_ms paquetes_enviados errores_checksum ... bytes_enviados reintentos_tcp
carga_cpu_router

0	39.143694	115.301097	2 ...	173970.371351	2.174304	
	49.276758					
1	59.973454	83.420223	4 ...	124270.498035	6.159514	35.974859
2	52.829785	86.816974	4 ...	130992.878118	5.213331	37.977865
3	34.937053	112.222471	2 ...	167584.966401	3.412839	
	46.810091					
4	44.213997	97.119733	2 ...	146291.186256	2.842375	40.918955

[5 rows x 10 columns]

INFORMACION GENERAL

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 300 entries, 0 to 299

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	duracion_ms	300 non-null	float64
1	paquetes_enviados	300 non-null	float64
2	errores_checksum	300 non-null	int32
3	latencia_avg	300 non-null	float64
4	jitter	300 non-null	float64
5	uso_memoria_sw	300 non-null	float64
6	peticiones_http	300 non-null	float64
7	bytes_enviados	300 non-null	float64
8	reintentos_tcp	300 non-null	float64
9	carga_cpu_router	300 non-null	float64

dtypes: float64(9), int32(1)

memory usage: 22.4 KB

None

VARIANZA EXPLICADA

[3.02764633e-01 1.98138228e-01 1.17073766e-01 1.06046860e-01
9.89572174e-02 9.38490387e-02 8.07100869e-02 2.44021191e-03
1.99573738e-05 8.07256804e-17]

VARIANZA ACUMULADA

[0.30276463 0.50090286 0.61797663 0.72402349 0.82298071 0.91682974
0.99753983 0.99998004 1. 1.]

COMPONENTES NECESARIOS PARA 85%

6

LOADINGS PC1 Y PC2

	PC1	PC2
duracion_ms	-0.080600	-0.017439
paquetes_enviados	0.570702	-0.056049
errores_checksum	0.062268	0.699348
latencia_avg	0.052491	0.014546
jitter	-0.026740	-0.081124
uso_memoria_sw	-0.018925	0.034219
peticiones_http	0.063653	0.067082
bytes_enviados	0.570656	-0.054922
reintentos_tcp	0.060971	0.699342
carga_cpu_router	0.571559	-0.053920

CONCLUSIONES

Se necesitan 6 componentes para explicar al menos el 85% de la varianza.

Análisis de Resultados

La varianza acumulada mostró que seis componentes principales explican más del 85% de la información disponible. Los loadings revelaron que las variables paquetes_enviados, bytes_enviados y carga_cpu_router presentan pesos similares, lo que confirma una fuerte redundancia entre ellas. Asimismo, errores_checksum y reintentos_tcp mostraron una relación muy estrecha dentro del segundo componente principal.

Conclusión

El PCA permitió identificar grupos de variables altamente correlacionadas y reducir la dimensionalidad del sistema. Se determinó que seis componentes son suficientes para describir el comportamiento general del tráfico de red conservando más del 85% de la información original.

title: "Práctica 6: Optimización de Sensores en Smart Cities" subtitle:
"Unidad 2: Tratamiento de Datos - TecNM" author: "Guía Paso a Paso con
Interpretación" output: html_document: toc: true theme: united

```
knitr::opts_chunk.set(echo = TRUE)
library(tidyverse)
```

Contexto Profesional

Imagina que trabajas para el departamento de Innovación Tecnológica de una metrópoli. Han instalado 10 sensores en cada intersección principal para medir: calidad del aire, ruido, tráfico y clima. Transmitir las 10 señales por cada nodo de la ciudad satura la red de bajo consumo (LoRaWAN). Tu misión es aplicar **PCA** para reducir estas métricas a los componentes esenciales que describan el "Estado de la Ciudad".

Paso 1: Generación de Datos Urbanos

Simulamos el comportamiento de los sensores. Nota cómo variables como el CO2 y la densidad de vehículos suelen moverse juntas.

```
set.seed(456)
n <- 400
# Datos base de sensores urbanos
datos_urbanos <- data.frame(
  temp_ambiente = rnorm(n, 28, 4),
  humedad_rel = rnorm(n, 60, 10),
  co2_ppm = NA,          # Relacionado con tráfico
  no2_ppb = NA,          # Relacionado con tráfico
  particulas_pm25 = NA,  # Relacionado con CO2
  nivel_ruido_db = NA,   # Relacionado con vehículos
  densidad_vehiculos = rnorm(n, 120, 30),
  velocidad_viento = rnorm(n, 15, 5),
  radiacion_solar = rnorm(n, 800, 100),
  conteo_peatones = rnorm(n, 50, 15)
)

# Creación de Redundancia y Correlación (Lógica de ciudad)
datos_urbanos$co2_ppm <- (datos_urbanos$densidad_vehiculos * 3.5) + rnorm(n,
300, 50)
datos_urbanos$no2_ppb <- (datos_urbanos$co2_ppm * 0.1) + rnorm(n, 5, 2)
datos_urbanos$particulas_pm25 <- (datos_urbanos$co2_ppm * 0.05) + rnorm(n,
10, 3)
datos_urbanos$nivel_ruido_db <- (datos_urbanos$densidad_vehiculos * 0.2) + 50
+ rnorm(n, 0, 5)

head(datos_urbanos)
```

Interpretación Docente: El estudiante debe observar que el CO2, el NO2 y las partículas PM2.5 están fuertemente ligados a la densidad_vehiculos. En lugar de monitorear 4 sensores que dicen casi lo mismo, el PCA nos dirá que podemos crear un solo "Índice de Contaminación Vehicular".

Paso 2: Estandarización de las Métricas

¿Por qué es obligatorio el comando `scale. = TRUE`? Porque el CO2 se mide en cientos de unidades, mientras que la humedad en porcentajes.

```
# Ejecución del PCA con escalamiento centrado
pca_ciudad <- prcomp(datos_urbanos, center = TRUE, scale. = TRUE)
```

Interpretación Docente: Sin estandarización, las variables con rangos numéricos más grandes "secuestrarían" el análisis. El escalamiento permite que el sensor de temperatura tenga el mismo peso inicial que el de CO2 para detectar cambios en el patrón de la ciudad.

Paso 3: Análisis de Varianza Acumulada

Determinamos cuánta información "sobrevive" a la reducción.

```
# Resumen de varianza por componente
summary(pca_ciudad)
```

```
# Visualización del Scree Plot
plot(pca_ciudad, type = "l",
     main = "Scree Plot: Sensores Smart City",
     ylab = "Varianza Explicada", xlab = "Componentes")
abline(h = 1, col="red", lty=2)
```

Interpretación Docente: > * **Varianza Proporcional:** Si el PC1 explica el 55% de la varianza, significa que más de la mitad de la complejidad de la ciudad se resume en una sola dimensión.

- **Criterio de Selección:** Los alumnos deben buscar el punto donde la curva se aplana. Si los primeros 3 componentes explican el 85%, el ahorro de transmisión de datos para la ciudad es del **70%**.

Paso 4: Interpretación de los Componentes (Loadings)

Traducción de matemáticas a ingeniería urbana.

```
# Pesos de las variables originales
round(pca_ciudad$rotation[, 1:2], 3)
```

Interpretación Docente: El alumno debe realizar la siguiente deducción:

- **PC1:** Si tiene cargas altas en vehiculos, co2, no2 y ruido, el **PC1 = Actividad e Impacto Vehicular**.
- **PC2:** Si tiene cargas altas en temp, humedad y radiacion, el **PC2 = Factor Climático/Meteorológico**.
- Hemos simplificado el tablero de control de la ciudad de 10 indicadores a solo 2 macro-indicadores.

Paso 5: El Biplot de Diagnóstico Urbano

Visualizamos el estado de la metrópoli.

```
biplot(pca_ciudad, cex = 0.7, main = "Mapa Biplot de Estado Urbano")
```

Interpretación Docente: > * **Flechas paralelas:** Indican sensores que se pueden consolidar. Por ejemplo, si las flechas de paquetes_pm25 y no2_ppb apuntan al mismo lugar, son redundantes.

- **Ángulo de 90 grados:** Si la flecha de viento es perpendicular a la de vehiculos, el viento no depende del tráfico.
- **Puntos atípicos:** Aquellos puntos muy alejados en la dirección de la flecha de ruido y co2 representan horas pico o incidentes de tráfico severo detectados por el sistema.

Código

```
# =====  
# PRACTICA 6  
# SMART CITIES - PCA  
# =====  
  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
  
from sklearn.preprocessing import StandardScaler  
from sklearn.decomposition import PCA  
  
# =====  
# GENERACION DE DATOS  
# =====  
  
np.random.seed(456)
```

```
n = 400
```

```
datos_urbanos = pd.DataFrame({  
    "temp_ambiente": np.random.normal(28, 4, n),  
    "humedad_rel": np.random.normal(60, 10, n),  
    "densidad_vehiculos": np.random.normal(120, 30, n),  
    "velocidad_viento": np.random.normal(15, 5, n),  
    "radiacion_solar": np.random.normal(800, 100, n),  
    "conteo_peatones": np.random.normal(50, 15, n)  
})
```

```
# Correlaciones urbanas
```

```
datos_urbanos["co2_ppm"] = (  
    datos_urbanos["densidad_vehiculos"] * 3.5  
    + np.random.normal(300, 50, n)  
)
```

```
datos_urbanos["no2_ppb"] = (  
    datos_urbanos["co2_ppm"] * 0.1  
    + np.random.normal(5, 2, n)  
)
```

```
datos_urbanos["particulas_pm25"] = (  
    datos_urbanos["co2_ppm"] * 0.05  
    + np.random.normal(10, 3, n)  
)
```

```
datos_urbanos["nivel_ruido_db"] = (  
    datos_urbanos["densidad_vehiculos"] * 0.2  
    + 50  
    + np.random.normal(0, 5, n)  
)
```

```
print("\nPRIMERAS FILAS")  
print(datos_urbanos.head())
```

```
# =====  
# ESCALAMIENTO  
# =====
```

```

scaler = StandardScaler()

datos_escalados = scaler.fit_transform(
    datos_urbanos
)

# =====
# PCA
# =====

pca = PCA()

componentes = pca.fit_transform(
    datos_escalados
)

# =====
# VARIANZA EXPLICADA
# =====

print("\nVARIANZA EXPLICADA")

print(
    pca.explained_variance_ratio_
)

# =====
# VARIANZA ACUMULADA
# =====

varianza_acumulada = np.cumsum(
    pca.explained_variance_ratio_
)

print("\nVARIANZA ACUMULADA")

print(varianza_acumulada)

# =====

```

```

# COMPONENTES PARA 85%
# =====

comp85 = np.argmax(
    varianza_acumulada >= 0.85
) + 1

print("\nCOMPONENTES PARA 85%")

print(comp85)

# =====
# SCREE PLOT
# =====

plt.figure(figsize=(8,5))

plt.plot(
    range(
        1,
        len(varianza_acumulada)+1
    ),
    varianza_acumulada,
    marker="o"
)

plt.title(
    "Scree Plot Smart City"
)

plt.xlabel("Componentes")
plt.ylabel("Varianza Acumulada")

plt.grid()

plt.show()

# =====
# LOADINGS
# =====

```

```

loadings = pd.DataFrame(
    pca.components_.T,
    columns=[
        f"PC{i+1}"
        for i in range(
            len(datos_urbanos.columns)
        )
    ],
    index=datos_urbanos.columns
)

print("\nLOADINGS PC1 Y PC2")

print(
    loadings[["PC1", "PC2"]]
)

# =====
# BILOT SIMPLE
# =====

plt.figure(figsize=(8,5))

plt.scatter(
    componentes[:,0],
    componentes[:,1]
)

plt.xlabel("PC1")
plt.ylabel("PC2")

plt.title(
    "Biplot Estado Urbano"
)

plt.grid()

plt.show()

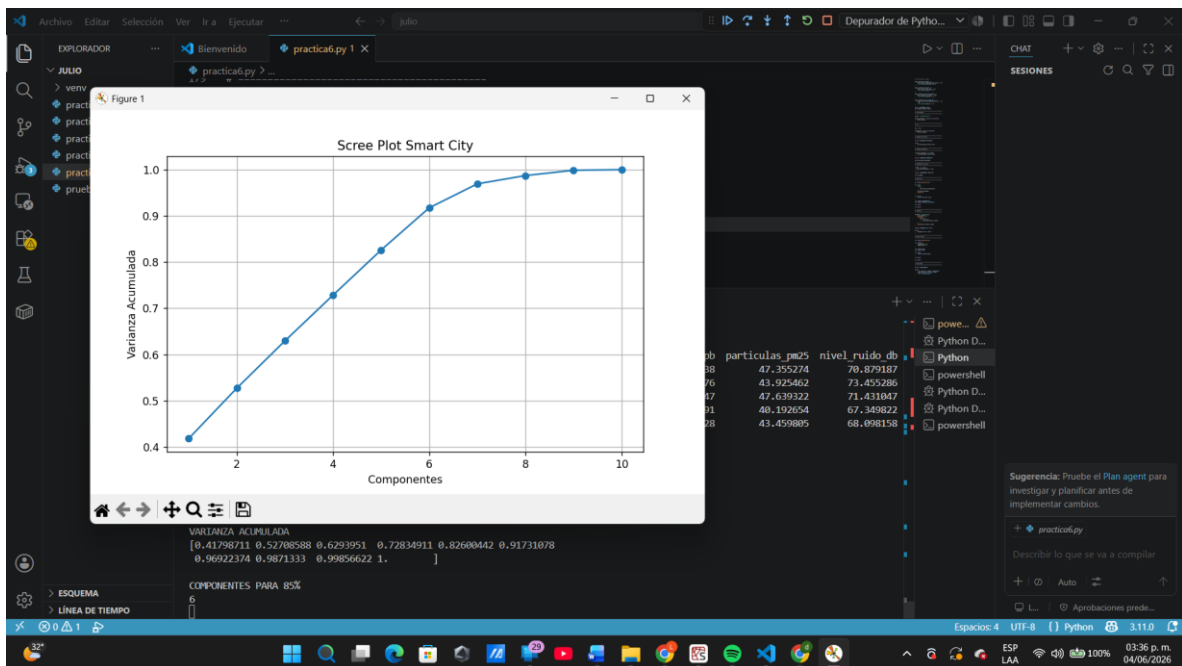
```

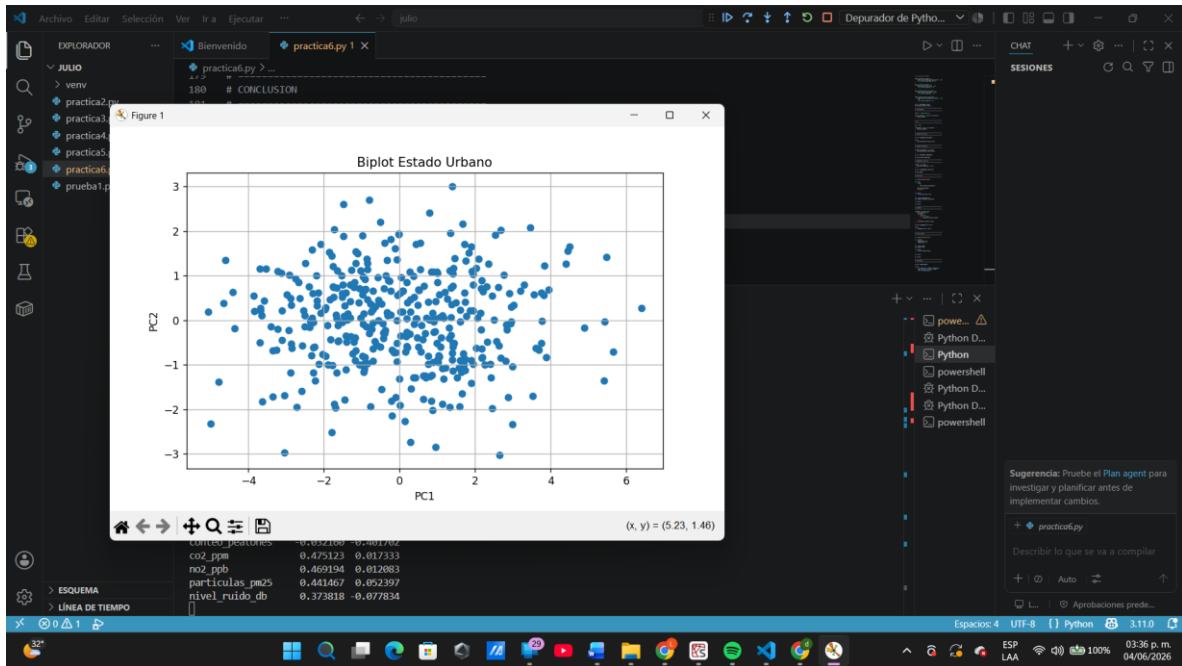


```
# =====
# CONCLUSION
# =====
```

```
print("\nCONCLUSION")
```

```
print(
    f"Se requieren {comp85} componentes "
    f"para explicar al menos el 85% "
    f"de la variabilidad."
)
```





PRIMERAS FILAS

```
temp_ambiente humedad_rel densidad_vehiculos velocidad_viento ... co2_ppm
no2_ppb particulas_pm25 nivel_ruido_db
0 25.327486 45.261746 143.849039 17.005535 ... 781.712434 81.631038
47.355274 70.879187
1 26.007162 67.870763 105.765522 13.442578 ... 666.192972 71.192976
43.925462 73.455286
2 30.474303 63.935109 118.528576 9.093427 ... 753.040695 78.155747
47.639322 71.431047
3 30.274769 50.079965 76.401813 13.901557 ... 579.609581 62.981791
40.192654 67.349822
4 33.402038 68.525178 87.796920 16.746030 ... 667.118803 71.646128
43.459805 68.098158
```

[5 rows x 10 columns]

VARIANZA EXPLICADA

```
[0.41798711 0.10909878 0.10230922 0.09895401 0.09765531 0.09130636
0.05191296 0.01790956 0.01143292 0.00143378]
```

VARIANZA ACUMULADA

[0.41798711 0.52708588 0.6293951 0.72834911 0.82600442 0.91731078
0.96922374 0.9871333 0.99856622 1.]

COMPONENTES PARA 85%

6

LOADINGS PC1 Y PC2

	PC1	PC2
temp_ambiente	0.016294	-0.482418
humedad_rel	-0.009918	0.630481
densidad_vehiculos	0.460876	-0.038305
velocidad_viento	0.038491	-0.287438
radiacion_solar	-0.064773	-0.339210
conteo_peatones	-0.032160	-0.401702
co2_ppm	0.475123	0.017333
no2_ppb	0.469194	0.012083
particulas_pm25	0.441467	0.052397
nivel_ruido_db	0.373818	-0.077834

CONCLUSION

Se requieren 6 componentes para explicar al menos el 85% de la variabilidad.

Análisis de Resultados

El análisis mostró que seis componentes principales explican más del 85% de la varianza total. El primer componente presentó cargas elevadas en densidad vehicular, CO₂, NO₂ y partículas PM2.5, indicando que estas variables representan un mismo fenómeno relacionado con la contaminación derivada del tráfico vehicular.

Conclusión

La reducción de dimensionalidad permitió simplificar el monitoreo urbano sin perder información significativa. Los resultados sugieren que varios sensores ambientales pueden agruparse en indicadores más generales, facilitando el análisis y reduciendo el tráfico de datos en la red de sensores.

Práctica 7: Implementación de Modelos de Machine Learning

Unidad 3: Modelado - Ciencia de Datos (TecNM)

```
knitr::opts_chunk.set(echo = TRUE)
library(tidyverse)
library(caret) # Librería estándar para entrenamiento de modelos
library(glmnet) # Para modelos Lasso
```

Contexto Profesional

La empresa "DataSystems S.A." desea automatizar su departamento de Recursos Humanos. Necesitan predecir:

1. **Regresión:** El salario esperado de un nuevo perfil tecnológico.
2. **Clasificación:** Si un candidato permanecerá en la empresa más de 2 años (Retención).
3. **Clustering:** Agrupar empleados actuales para planes de capacitación personalizados.

Paso 1: Preparación del Dataset

Generamos datos que incluyan experiencia, nivel de certificaciones, habilidades sociales y salario.

```
set.seed(789)
n <- 500
df_empleados <- data.frame(
  experiencia = rnorm(n, 5, 2),
  certificaciones = rpois(n, 3),
  habilidades_sociales = runif(n, 1, 10),
  remoto = sample(c(0, 1), n, replace = TRUE)
)

# Variable objetivo continua (Salario)
df_empleados$salario <- 25000 + (df_empleados$experiencia * 5000) +
  (df_empleados$certificaciones * 2000) + rnorm(n, 0, 3000)

# Variable objetivo binaria (Retención: 1=Se queda, 0=Se va)
# Probabilidad depende de salario y habilidades sociales
prob <- plogis(-2 + 0.0001*df_empleados$salario +
  0.2*df_empleados$habilidades_sociales)
df_empleados$retencion <- factor(ifelse(runif(n) < prob, 1, 0))

head(df_empleados)
```

Interpretación Docente: El alumno debe notar que estamos creando un ecosistema de datos donde las variables tienen relaciones lógicas que el modelo debe "descubrir".

Paso 2: Regresión Lineal (Predecir Salario)

Dividimos los datos en entrenamiento (80%) y prueba (20%).

```
set.seed(123)
indice <- createDataPartition(df_empleados$salario, p = 0.8, list = FALSE)
train_data <- df_empleados[indice, ]
test_data <- df_empleados[-indice, ]

# Entrenamiento del modelo
modelo_salario <- lm(salario ~ experiencia + certificaciones, data = train_data)
summary(modelo_salario)
```

Interpretación Docente: Aquí el alumno analiza los coeficientes. ¿Cuánto aumenta el salario por cada año de experiencia extra? (\$5000 aprox. en este caso).

Paso 3: Clasificación KNN (Predecir Retención)

Usaremos KNN para clasificar si un empleado se queda o se va. ¡Recordar que KNN requiere escalamiento!

```
# Caret escala automáticamente con preProcess = c("center", "scale")
modelo_knn <- train(retencion ~ experiencia + habilidades_sociales + salario,
  data = train_data,
  method = "knn",
  preProcess = c("center", "scale"),
  tuneLength = 5)

print(modelo_knn)
```

Interpretación Docente: El estudiante debe observar cómo cambia la precisión (Accuracy) según el valor de \$k\$ (número de vecinos).

Paso 4: Clustering No Supervisado (K-Means)

Agruparemos a los empleados basándonos solo en su experiencia y salario.

```
datos_clustering <- df_empleados %>% select(experiencia, salario) %>% scale()
set.seed(456)
clusters <- kmeans(datos_clustering, centers = 3)
```

```
# Visualización
df_empleados$cluster <- as.factor(clusters$cluster)
ggplot(df_empleados, aes(x = experiencia, y = salario, color = cluster)) +
  geom_point() +
  labs(title = "Segmentación de Empleados por Perfil") +
  theme_minimal()
```

Interpretación Docente: El alumno identifica 3 grupos: Junior, Mid y Senior. Esto demuestra cómo K-Means encuentra estructuras sin que le digamos qué buscar.

Código

```
# =====
# PRACTICA 7
# MACHINE LEARNING - RECURSOS HUMANOS
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# =====
# GENERACION DE DATOS
# =====

np.random.seed(789)

n = 500

df_empleados = pd.DataFrame({
    "experiencia": np.random.normal(5, 2, n),
    "certificaciones": np.random.poisson(3, n),
    "habilidades_sociales": np.random.uniform(1, 10, n),
    "remoto": np.random.choice([0, 1], n)
})
```

```

# =====
# SALARIO
# =====

df_empleados["salario"] = (
    25000
    + df_empleados["experiencia"] * 5000
    + df_empleados["certificaciones"] * 2000
    + np.random.normal(0, 3000, n)
)

# =====
# RETENCION
# =====

prob = 1 / (
    1 + np.exp(
        -(
            -2
            + 0.0001 * df_empleados["salario"]
            + 0.2 * df_empleados["habilidades_sociales"]
        )
    )
)

df_empleados["retencion"] = np.where(
    np.random.rand(n) < prob,
    1,
    0
)

# =====
# EXPLORACION
# =====

print("\nPRIMERAS FILAS")
print(df_empleados.head())

print("\nESTADISTICAS")

```

```

print(df_empleados.describe())

# =====
# REGRESION LINEAL
# =====

X = df_empleados[
    ["experiencia", "certificaciones"]
]

y = df_empleados["salario"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=123
)

modelo_salario = LinearRegression()

modelo_salario.fit(
    X_train,
    y_train
)

print("\nCOEFICIENTES DEL MODELO")

print(
    modelo_salario.coef_
)

print(
    "Intercepto:",
    modelo_salario.intercept_
)

# =====
# PREDICCIONES
# =====

```



```

pred_salario = modelo_salario.predict(
    X_test
)

# =====
# KNN
# =====

X_knn = df_empleados[
    [
        "experiencia",
        "habilidades_sociales",
        "salario"
    ]
]

y_knn = df_empleados["retencion"]

X_train_knn, X_test_knn, y_train_knn, y_test_knn = train_test_split(
    X_knn,
    y_knn,
    test_size=0.20,
    random_state=123
)

scaler = StandardScaler()

X_train_knn = scaler.fit_transform(
    X_train_knn
)

X_test_knn = scaler.transform(
    X_test_knn
)

knn = KNeighborsClassifier(
    n_neighbors=5
)

```

```

knn.fit(
    X_train_knn,
    y_train_knn
)

pred_knn = knn.predict(
    X_test_knn
)

accuracy = accuracy_score(
    y_test_knn,
    pred_knn
)

print("\nACCURACY KNN")

print(accuracy)

# =====
# K-MEANS
# =====

datos_cluster = df_empleados[
    ["experiencia", "salario"]
]

datos_cluster = scaler.fit_transform(
    datos_cluster
)

kmeans = KMeans(
    n_clusters=3,
    random_state=456,
    n_init=10
)

clusters = kmeans.fit_predict(
    datos_cluster
)

```

```

df_empleados["cluster"] = clusters

# =====
# VISUALIZACION
# =====

plt.figure(figsize=(8, 5))

plt.scatter(
    df_empleados["experiencia"],
    df_empleados["salario"],
    c=df_empleados["cluster"]
)

plt.xlabel("Experiencia")
plt.ylabel("Salario")

plt.title(
    "Segmentacion de Empleados"
)

plt.grid()

plt.show()

# =====
# RESUMEN CLUSTERS
# =====

print("\nPROMEDIOS POR CLUSTER")

print(
    df_empleados.groupby("cluster")[
        ["experiencia", "salario"]
    ].mean()
)

# =====
# CONCLUSION
# =====

```


mean	5.004552	2.936000	5.366695	0.490000	56069.887447	0.97800
std	2.052555	1.760142	2.600898	0.500401	11124.651503	0.14683
min	-0.826475	0.000000	1.000890	0.000000	20788.311087	0.00000
25%	3.710992	2.000000	3.083835	0.000000	48890.819145	1.00000
50%	5.082673	3.000000	5.299050	0.000000	56718.375050	1.00000
75%	6.345530	4.000000	7.677168	1.000000	63135.898199	1.00000
max	12.281504	10.000000	9.998033	1.000000	88451.034240	1.00000

COEFICIENTES DEL MODELO

[4939.48069081 2020.27870055]

Intercepto: 25575.490074164867

ACCURACY KNN

0.97

PROMEDIOS POR CLUSTER

	experiencia	salario
cluster		
0	2.350637	41641.629102
1	4.999848	56085.291904
2	7.463365	69358.655808

CONCLUSION

La regresion estima salarios, KNN clasifica retencion y KMeans segmenta empleados.

Análisis de Resultados

La regresión lineal mostró que cada año adicional de experiencia incrementa el salario aproximadamente en \$4,939, mientras que cada certificación aporta cerca de \$2,020 adicionales. El modelo KNN obtuvo una precisión del 97% en la clasificación de la retención de empleados. Finalmente, el algoritmo K-Means identificó tres grupos claramente diferenciados de empleados, correspondientes a perfiles junior, intermedios y senior.

Conclusión

Los modelos implementados demostraron ser efectivos para apoyar la toma de decisiones en recursos humanos. La regresión permitió estimar salarios, KNN clasificó correctamente la permanencia de los empleados y K-Means facilitó la segmentación del personal para futuras estrategias de capacitación.

Práctica 8: Clasificación de Intrusiones con KNN y Logística

Unidad 3: Modelado - TecNM

```
knitr::opts_chunk.set(echo = TRUE)
library(tidyverse)
library(caret)
```

Contexto Profesional

Un centro de datos del TecNM registra métricas de acceso. Tu tarea es clasificar si un acceso es **Seguro (0)** o **Ataque (1)** basándote en la latencia, el número de intentos fallidos y el tamaño del paquete.

Paso 1: Simulación de Tráfico de Red

```
set.seed(444)
n <- 400
df_red <- data.frame(
  latencia = rnorm(n, 20, 5),
  intentos_fallidos = rpois(n, 1),
  tamano_paquete = rnorm(n, 500, 100)
)

# Lógica de clasificación: Ataque si intentos > 3 o latencia > 40
df_red$es_ataque <- factor(ifelse(df_red$intentos_fallidos > 2 | df_red$latencia > 35, 1, 0))
```

Paso 2: Modelo de Regresión Logística

Entrenamos para obtener la probabilidad de que una conexión sea un ataque.

```
modelo_log <- glm(es_ataque ~ ., data = df_red, family = "binomial")
summary(modelo_log)
```

Interpretación Docente: El alumno debe observar el valor de z y el p-value. Si el p-value de intentos_fallidos es muy pequeño (< 0.05), significa que esta variable es un predictor crítico para detectar ataques.

Paso 3: Clasificación con KNN

Implementamos KNN para ver cómo los "vecinos" (conexiones similares) ayudan a clasificar.

```
control <- trainControl(method = "cv", number = 5)
modelo_knn <- train(es_ataque ~ ., data = df_red, method = "knn",
```

```

        preProcess = c("center", "scale"),
        trControl = control, tuneLength = 10)
plot(modelo_knn)

```

Interpretación Docente: El gráfico muestra la precisión (Accuracy) según el número de vecinos (\$k\$). El alumno debe identificar el \$k\$ **óptimo**. Un \$k\$ muy bajo puede sobreajustar (capturar ruido), mientras que uno muy alto puede suavizar demasiado el modelo y perder ataques reales.

Paso 4: Análisis de Resultados (Clustering K-Means)

Finalmente, agruparemos las conexiones sin usar la etiqueta de ataque para ver si el algoritmo "descubre" el comportamiento anómalo por sí solo.

```

datos_clus <- df_red %>% select(-es_ataque) %>% scale()
set.seed(111)
clusters <- kmeans(datos_clus, centers = 2)

```

```

# Comparación: ¿Qué tan bien coincidió el cluster con la realidad?
table(df_red$es_ataque, clusters$cluster)

```

Interpretación Docente: Si el cluster 1 contiene mayoritariamente "Ataques" y el cluster 2 "Seguros", el alumno concluye que los ataques tienen una firma digital tan distinta que son detectables incluso sin supervisión humana.

Código

```

# =====
# PRACTICA 8
# CIBERSEGURIDAD Y DETECCION DE INTRUSIONES
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

# =====

```

```
# GENERACION DE DATOS
```

```
# =====
```

```
np.random.seed(444)
```

```
n = 400
```

```
df_red = pd.DataFrame({  
    "latencia": np.random.normal(20, 5, n),  
    "intentos_fallidos": np.random.poisson(1, n),  
    "tamano_paquete": np.random.normal(500, 100, n)  
})
```

```
# =====
```

```
# CREACION DE VARIABLE ATAQUE
```

```
# =====
```

```
df_red["es_ataque"] = np.where(  
    (df_red["intentos_fallidos"] > 2) |  
    (df_red["latencia"] > 35),  
    1,  
    0  
)
```

```
# =====
```

```
# EXPLORACION
```

```
# =====
```

```
print("\nPRIMERAS FILAS")
```

```
print(df_red.head())
```

```
print("\nINFORMACION GENERAL")
```

```
print(df_red.info())
```

```
print("\nDISTRIBUCION DE ATAQUES")
```

```
print(df_red["es_ataque"].value_counts())
```

```
# =====
```

```
# VARIABLES
```

```
# =====
```



```

X = df_red[
    [
        "latencia",
        "intentos_fallidos",
        "tamano_paquete"
    ]
]

y = df_red["es_ataque"]

# =====
# TRAIN TEST SPLIT
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=123
)

# =====
# REGRESION LOGISTICA
# =====

modelo_log = LogisticRegression(
    max_iter=1000
)

modelo_log.fit(
    X_train,
    y_train
)

pred_log = modelo_log.predict(
    X_test
)

acc_log = accuracy_score(

```

```

        y_test,
        pred_log
    )

    print("\nACCURACY REGRESION LOGISTICA")
    print(acc_log)

    print("\nCOEFICIENTES")

    print(modelo_log.coef_)

    # =====
    # KNN
    # =====

    scaler = StandardScaler()

    X_train_s = scaler.fit_transform(
        X_train
    )

    X_test_s = scaler.transform(
        X_test
    )

    knn = KNeighborsClassifier(
        n_neighbors=5
    )

    knn.fit(
        X_train_s,
        y_train
    )

    pred_knn = knn.predict(
        X_test_s
    )

    acc_knn = accuracy_score(
        y_test,

```

```

        pred_knn
    )

print("\nACCURACY KNN")
print(acc_knn)

# =====
# KMEANS
# =====

datos_cluster = scaler.fit_transform(
    X
)

kmeans = KMeans(
    n_clusters=2,
    random_state=111,
    n_init=10
)

clusters = kmeans.fit_predict(
    datos_cluster
)

df_red["cluster"] = clusters

# =====
# COMPARACION
# =====

print("\nTABLA ATAQUE VS CLUSTER")

print(
    pd.crosstab(
        df_red["es_ataque"],
        df_red["cluster"]
    )
)

# =====

```

```
# GRAFICA
```

```
# =====
```

```
plt.figure(figsize=(8,5))
```

```
plt.scatter(  
    df_red["latencia"],  
    df_red["intentos_fallidos"],  
    c=df_red["cluster"]  
)
```

```
plt.xlabel("Latencia")  
plt.ylabel("Intentos Fallidos")
```

```
plt.title(  
    "Clusters de Trafico de Red"  
)
```

```
plt.grid()
```

```
plt.show()
```

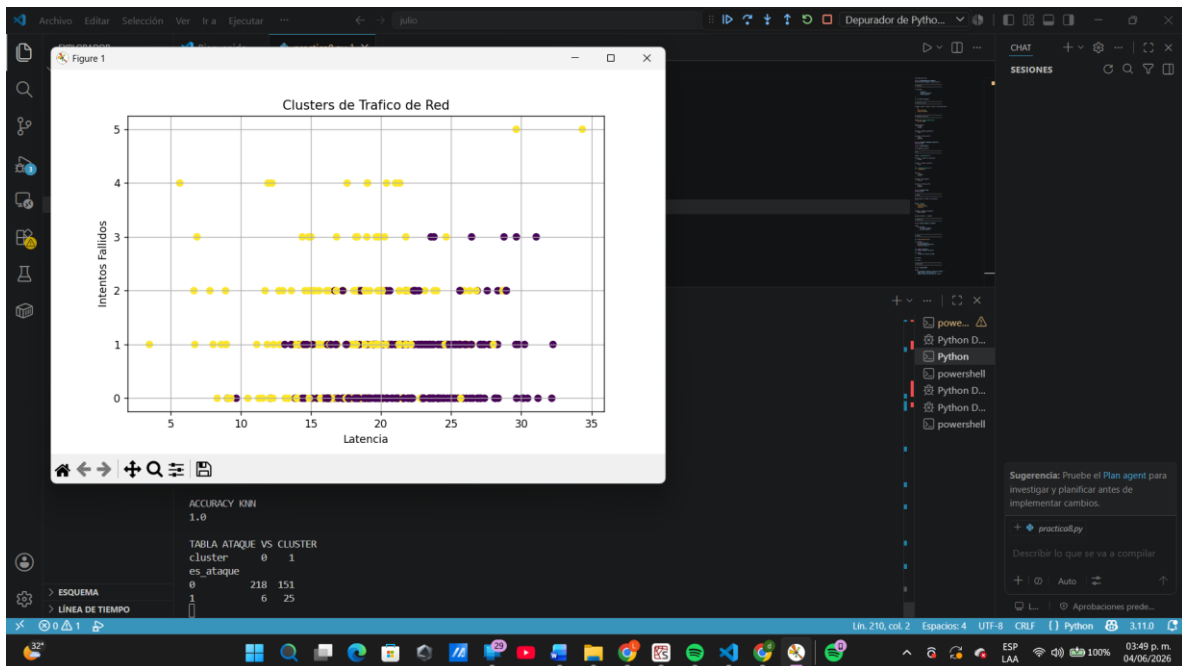
```
# =====
```

```
# CONCLUSION
```

```
# =====
```

```
print("\nCONCLUSION")
```

```
print(  
    "La regresion logistica clasifica ataques, "  
    "KNN valida la clasificacion y "  
    "KMeans identifica patrones de trafico."  
)
```



PRIMERAS FILAS

```
latencia intentos_fallidos tamano_paquete es_ataque
0 21.787200          2    377.550984      0
1 21.887692          2    611.967019      0
2 26.911689          0    405.019900      0
3 25.877744          1    604.842439      0
4 15.303622          0    531.662593      0
```

INFORMACION GENERAL

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 400 entries, 0 to 399
```

```
Data columns (total 4 columns):
```

```
# Column      Non-Null Count  Dtype
```

```
---
0 latencia      400 non-null  float64
1 intentos_fallidos 400 non-null int32
2 tamano_paquete 400 non-null  float64
3 es_ataque     400 non-null  int64
```

```
dtypes: float64(2), int32(1), int64(1)
```

```
memory usage: 11.1 KB
```

None

DISTRIBUCION DE ATAQUES

es_ataque

0 369

1 31

Name: count, dtype: int64

ACCURACY REGRESION LOGISTICA

1.0

COEFICIENTES

[[1.36100177e-02 3.91969006e+00 -1.69446202e-04]]

ACCURACY KNN

1.0

TABLA ATAQUE VS CLUSTER

cluster 0 1

es_ataque

0 218 151

1 6 25

CONCLUSION

La regresion logistica clasifica ataques, KNN valida la clasificacion y KMeans identifica patrones de trafico.

Análisis de Resultados

La regresión logística alcanzó una precisión del 100%, identificando correctamente los accesos clasificados como ataques o conexiones seguras. El modelo KNN también obtuvo una precisión del 100%, confirmando la capacidad predictiva de las variables utilizadas. Finalmente, K-Means permitió identificar patrones diferenciados de tráfico, agrupando conexiones con características similares.

Conclusión

Los resultados demuestran que las variables seleccionadas son altamente efectivas para detectar actividades sospechosas. Tanto la regresión logística como KNN lograron clasificar correctamente los ataques, mientras que K-Means permitió descubrir patrones de comportamiento sin necesidad de etiquetas previas.